

**'OPCControlTemplateR0': Use and Function****12/1/2022****Rev. 1****ABSTRACT:**

This document contains all information necessary to allow users of the MATLAB program 'OPCControlTemplateR0' competence to a level that: (a) The program can be used in its default state to enable PI control of the configured controllers over OPC. (b) The program can be modified to allow the construction of control schemes not natively supported by the EPKS control builder. The document has three main sections; the first covers a quick start guide of the program, including navigation and use of the program. The second section covers a more extensive description of the program's function and operation process. Finally, a full copy of the code is offered to ensure that an unmodified version of the program is always available.

Contents

List of Tables	3
List of Figures	3
Document Control and Revision History.....	4
1.0 Definitions.....	5
2.0 Purpose	6
3.0 Scope.....	7
4.0 Quick Start Guide	8
4.1 Navigating the Program	8
4.2 Use of the Program in its Default State	10
4.3 Modification of Program Control Schemes.....	12
5.0 Extensive Program Description	15
5.1 I. Initialization.....	15
1. Server Connect.....	15
2a. Control Group Establishment.....	15
2b. Bonus Variable Group Establishment	16
3a. Variable Creation, Initialization, and Server Linking	16
3b. Bonus Variables Creation and First Read	17
5.2 II. Main Loop (Controllers)	18
3c. Bonus Variables n^{th} Read	18
4-23: Controllers 1-20	18
6.0 'OPCControlTemplateR0' – Code	22

List of Tables

Table 1: Document Control and Revision History	4
Table 2: Summary of sections within the ' <i>OPCControlTemplateR0</i> ' program, italicized red text indicates potential points of modification for program users.	9

List of Figures

Figure 1: 'Go to section' selector on MATLAB editor ribbon.	8
Figure 2: Block diagram of the designed controller and simplified physical system: Control flow can be configured from EPKS HMI such that MATLAB achieves outer-loop cascade control via OPC (<i>a</i>) OR through EPKS CM (<i>b</i>). Outer-loop controller output is then fed into the inner-loop controller (<i>c</i>) housed in the EPKS CM, the output of which is fed into the physical system (<i>d</i>). <i>NOTE: Outer-loop controllers can also be fed directly into the physical system, thus becoming the controller under a single-loop control scheme.</i>	10
Figure 3: Lines 654 to 657 of the ' <i>OPCControlTemplateR0</i> ' code.....	11
Figure 4: Feed Pump control popup, <i>a</i> shows OPC enable tick box, enabling this allows routing out of EPKS via OPC; <i>b</i> shows the OPC Shared Variables, these values can be read in an OPC compliant program, allowing configurable control parameters to be easily controlled from the EPKS HMI.	11
Figure 5: Lines 671 to 705 of the ' <i>OPCControlTemplateR0</i> ' code. The figure shows first the n^{th} read of controller 1 variables from the OPC server, then the configurable section of the feed pump controller set up to act as a simple discrete PI controller. OPC-shared variables A and B are imported from EPKS to act as controller gain and integrator time.	12
Figure 6: Controller 1 controller section; intended for modification by program users.	14
Figure 7: Section 1 of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.....	15
Figure 8: Section 2a of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.....	15
Figure 9: Section 2b of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.	16
Figure 10: An excerpt of section 3a of the ' <i>OPCControlTemplateR0</i> ' program. The figure includes initialization of program variables <i>n</i> , and <i>t</i> , the start of the first program timer, and creation and initialization of controller 1 variables.	17
Figure 11: Section 3b of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.	17
Figure 12: Program main loop, initialization and housekeeping.	18
Figure 13: Section 3c of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.....	18
Figure 14: Section 4 of the ' <i>OPCControlTemplateR0</i> ' MATLAB program.....	20

Document Control and Revision History

Table 1: Document Control and Revision History

<i>Revision</i>	<i>Date</i>	<i>Author(s)</i>	<i>Reviewed by</i>	<i>Comments/Details</i>
0	2022	Nicholas O’Keeffe		

Definitions

K_c	Controller Gain
τ_I	Integral Time
BM	Ball Mill
BMP	Ball Mill Pump
BMT	Ball Mill Tank
CRP	Cyclone Recycle Pump
CSTR	Continuously Stirred Tank Reactor
CUPP	Cyclone Underflow Pump
DV	Disturbance Variable
EPKS	Experion Process Knowledge System
FCV	Flow Control Valve
FDP	Feed Disturbance Pump
FP	Feed Pump
GMC	Generic Model Control
HMI	Human Machine Interface
LUP	Lamella Underflow Pump
MUPP	Murdoch University Pilot Plant
MV	Manipulated Variable
NTP	Needle Tank Underflow Pump
OOO	Out of Commission
OPC	Open Platform Communications
OPC DA	Open Platform Communications, Data Access
PI	Proportional, Integral
PP	Product Pump
PV	Process Variable
QSG	Quick Start Guide
SP	Set Point
ε	Error

1.0 Purpose

This document's purpose lies in exploring and explaining the MATLAB program '*OPCControlTemplateR0*', which was developed as part of the MUPP control system overhaul carried out in semester 2 of 2022. '*OPCControlTemplateR0*' enables pain-free controller design for the MUPP over OPC. Functionality is achieved by integrating the program fully with the EPKS HMI, which was revised in tandem with this program. The integration is such that the MATLAB program can be used solely for the mathematical implementation of control schemes, with the tuning of control parameters, routing of process and disturbance variables, and control of the setpoint all being dictated within the EPKS HMI.

2.0 Scope

This document covers all information required for a user of the 'OPCControlTemplateR0' program to achieve competence, such that the user is capable of:

- Using the program in its default state, *the program, by default, can enable PI control via all the configured controllers.*
- Modifying the program to enable novel control schemes that are not natively available in the EPKS ecosystem.

Instruction to enable this competence is covered in two sections, first as a quick start guide which offers a crash course in the program. This section covers navigation of the program, use of the program in its default state, and modification of the program to allow customization of the controllers housed within. The second section extensively describes the program process, granting users a complete understanding of the program. The intent is that with a thorough knowledge of the program, future users will be able to enact more novel control schemes of the MUPP.

A complete copy of the code in its unmodified state is available at the back of this document should the original version of the program be lost.

3.0 Quick Start Guide

3.1 Navigating the Program

The “*OPCControlTemplateR0*” MATLAB program is extensive and, as a result, can appear to be quite intimidating. Fortunately, however, few sections of the code require modification from program users to achieve meaningful modifications to control architecture. In terms of program navigation, the easiest method is with the “Go To” section dropdown menu located in the editor ribbon of the MATLAB toolbar, which can be observed in Figure 1.

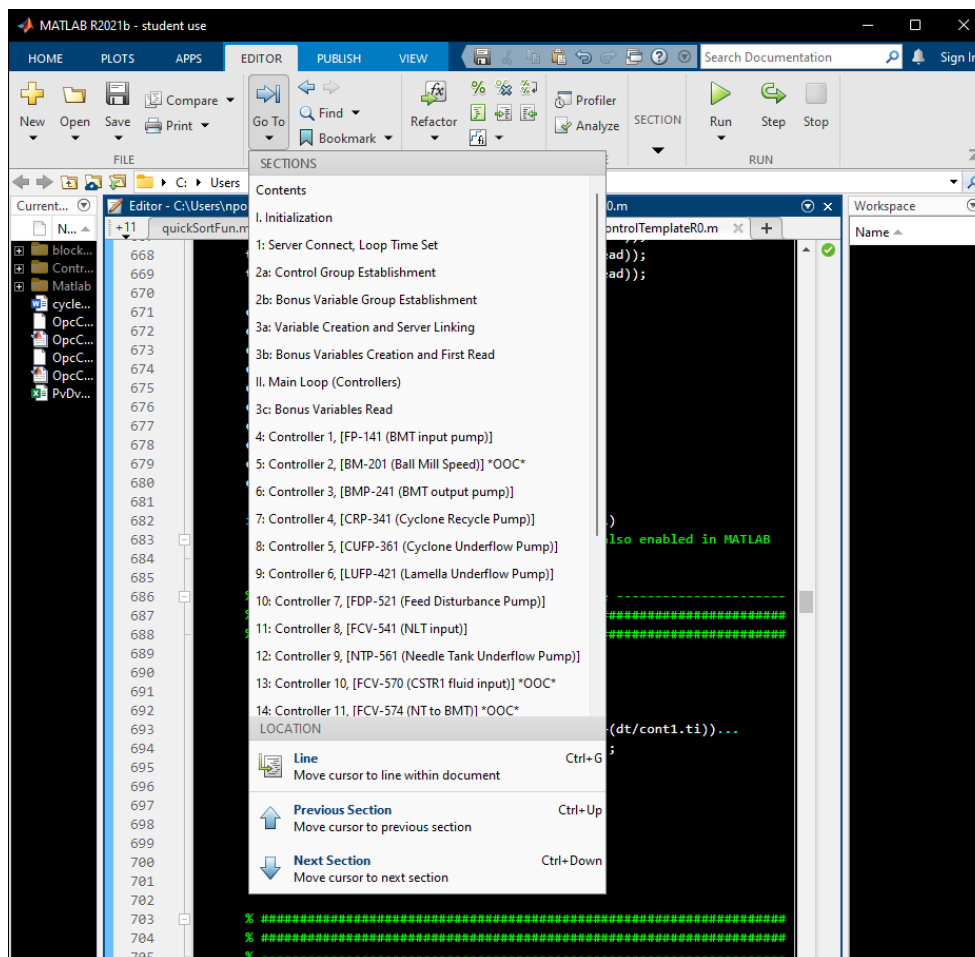


Figure 1: ‘Go to section’ selector on MATLAB editor ribbon.

For clarification on the context of each of the sections outlined in the “Go To” section dropdown menu, Table 2 should be consulted. Table 2 lists each program section and summarises its purpose; further, the notes in italicized red indicate potential points of program user modification for changes to the control strategy.

Table 2: Summary of sections within the 'OPCControlTemplateR0' program, italicized red text indicates potential points of modification for program users.

Section:	Context:
I. Initialization	
1: Server Connect, Loop Time Set	Defines and connects to OPC server and establishes controller loop time. <i>Program users can set loop time here; by default, this is set to 1 second.</i>
2a: Control Group Establishment	Defines OPC communication groups; with EPKS, these communication groups are synonymous with control modules; for example, the feed pump reference speed control module is 'FP_REF_141-R1A'.
2b: Bonus Variable Group Establishment	Defines additional OPC communication groups. <i>Program users can define additional OPC communications groups corresponding to additional variables that the user may require for the control scheme; for example, many additional parameters may be necessary for a generic model controller (GMC).</i>
3a: Variable Creation, Initialization and Server Linking	Defines point parameters for each controller and performs the initial read of said point parameters. Point parameters correspond to EPKS control module block inputs/outputs. Each controller has 11 associated point parameters: 1. The associated mv read from EPKS, 2, 3, 4, 5. The associated PV1, PV2, DV, and SP read from EPKS, respectively, 6, 7, 8, 9. OPC shared variables a, b, c, and d, read from EPKS and controlled from the EPKS HMI, 10. OPC enable Boolean read-from/write-to EPKS, and 11. The associated mv write to EPKS.
3b: Bonus Variables Creation and First Read	Definition and first read of additional OPC communication points. <i>This section is where program users can define the individual point parameters corresponding to the communication groups defined in section 2b.</i>
II. Main Loop Controllers	
3c: Bonus Variables Read	Main loop additional OPC communication point read. <i>This section is for the regular reading of the additional OPC communication points corresponding to the groups and point parameters defined in sections 2b and 3b, respectively.</i>
4: Controller 1, [FP-141 (BMT input pump)]	Main loop controller for feed pump (FP-141). <i>Configure the feed pump controller in this section.</i>
5: Controller 2, [BM-201 (Ball Mill Speed)]	Main loop controller for ball mill speed. <i>Out of Commission.</i>
6: Controller 3, [BMP-241 (BMT output pump)]	Main loop controller for ball mill pump (BMP-241). <i>Configure the ball mill pump controller here.</i>
7: Controller 4, [CRP-341 (Cyclone Recycle Pump)]	Main loop controller for cyclone recycle pump (CRP-341). <i>Configure the cyclone recycle pump controller here.</i>
8: Controller 5, [CUFP-361 (Cyclone Underflow Pump)]	Main loop controller for cyclone underflow pump (CUFP-361). <i>Configure the cyclone underflow pump controller here.</i>
9: Controller 6, [LUFP-421 (Lamella Underflow Pump)]	Main loop controller for lamella underflow pump (LUFP-421). <i>Unconfigured.</i>
10: Controller 7, [FDP-521 (Feed Disturbance Pump)]	Main loop controller for feed disturbance pump (FDP-521). <i>Configure the feed disturbance pump controller here.</i>
11: Controller 8, [FCV-541 (NLT input)]	Main loop controller for flow control valve input into the non-linear tank (FCV-541). <i>Configure flow control valve 541 controller here.</i>
12: Controller 9, [NTP-561 (Needle Tank Underflow Pump)]	Main loop controller for needle tank underflow pump (NTP-561). <i>Configure needle tank underflow pump controller here.</i>
13: Controller 10, [FCV-570 (CSTR1 input)] *OOC*	Main loop controller for flow control valve between needle tank underflow pump and CSTR-1 input (FCV-570). <i>Out of Commission.</i>
14: Controller 11, [FCV-574 (NT to BMT)] *OOC*	Main loop controller for flow control valve between needle tank and ball mill input (FCV-574). <i>Out of Commission.</i>
15: Controller 12, [FCV-622 (CSTR1 Steam In)] *OOC*	Main loop controller for steam input flow control valve into CSTR-1 (FCV-622). <i>Out of Commission.</i>

Section:	Context:
16: Controller 13, [FCV-642 (CSTR2 Steam In)]	Main loop controller for steam input flow control valve into CSTR-2 (FCV-642). <i>Configure flow control valve 642 controller here.</i>
17: Controller 14, [FCV-662 (CSTR3 Steam In)]	Main loop controller for steam input flow control valve into CSTR-3 (FCV-662). <i>Configure flow control valve 662 controller here.</i>
18: Controller 15, [FCV-688 (System Output)]	Main loop controller for flow control valve between product pump and system output (FCV-688). <i>Configure flow control valve 688 controller here.</i>
19: Controller 16, [FCV-690 (CSTR3 Reflux)]	Main loop controller for flow control valve between product pump and reflux into CSTRs 1, 2, and 3 (FCV-690). <i>Configure flow control valve 690 controller here.</i>
20: Controller 17, [PP-681 (Product Pump)]	Main loop controller for product pump (PP-681). <i>Configure the product pump controller here.</i>
21: Controller 18, [SPARE]	Spare controller; corresponds to PID18_PvSp_SELECT-R1A and PID18_AutVar_SELECT-R0A control modules.
22: Controller 19, [FCV-571 (NT to ST Bank)] *OOC*	Main loop controller for flow control valve between needle tank and storage tanks (FCV-571). <i>Out of Commission.</i>
23: Controller 20, [SPARE]	Spare controller; corresponds to PID20_PvSp_SELECT-R1A and PID20_AutVar_SELECT-R0A control modules.

3.2 Use of the Program in its Default State

In its default state, “OPCControlTemplateR0” is configured to allow PI control of each of the twelve manipulated variables configured for control in the EPKS control builder. Observing Figure 2, we can see the location of the MATLAB OPC program within the context of the entire control system. ***As the program exists as only a single element of the overall control architecture, it is worth remembering that the overall control scheme is still influenced by selections made on the EPKS HMI. When the program is active, the program can act as either the outer-loop controller in a cascade control scheme or the sole controller in a single-loop control scheme.***

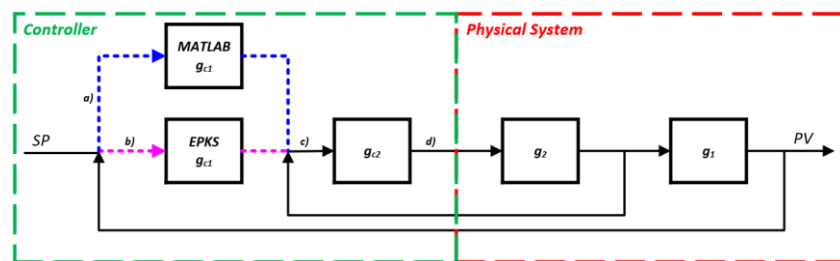


Figure 2: Block diagram of the designed controller and simplified physical system: Control flow can be configured from EPKS HMI such that MATLAB achieves outer-loop cascade control via OPC (a) OR through EPKS CM (b). Outer-loop controller output is then fed into the inner-loop controller (c) housed in the EPKS CM, the output of which is fed into the physical system (d). **NOTE: Outer-loop controllers can also be fed directly into the physical system, thus becoming the controller under a single-loop control scheme.**

The only change to the program required for immediate use of the program is the first line of each of the controllers that are desired for use. Figure 3 shows the line that must be changed in the first controller, belonging to the feed pump (FP-141). The purpose of this line is to block the routing of control through the MATLAB program when the controller is not configured. It does this by skipping the control segment of code for the controller and resetting the control Boolean in EPKS responsible for allowing the routing

of the OPC-controlled manipulated variable to the final control element. **To enable OPC control, change the value of 'MIOpcEnable' from false to true.**

```
%%          4: Controller 1, [FP-141 (BMT input pump)]
cont1.MIOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
% MATLAB, else 0 will be written to OPC server.
```

Figure 3: Lines 654 to 657 of the 'OPCControlTemplateR0' code.

Once the desired controllers have been enabled in the code, the program can be started. As routing of variables and controller parameters are configured from the HMI, the MATLAB program can be mostly ignored; it is wise to monitor the program to ensure it does not stop working.

With the MATLAB program running, OPC control is activated on a per-controller basis from the popup menu for each controller on the EPKS HMI. Figure 4 shows one such controller popup, the controller for FP-141; observing *a* in Figure 4, we can see the tick box for enabling OPC control; checking this box starts OPC control. Observing *b* in Figure 4, we can see the OPC shared variables; these are variables inside the "OPCControlTemplateR0" program, which are controlled from this section of the HMI. In this case, the shared variables have been configured such that OPC shared variable a corresponds to controller gain (K_c) and OPC shared variable b corresponds to integrator time (τ_I) in seconds.

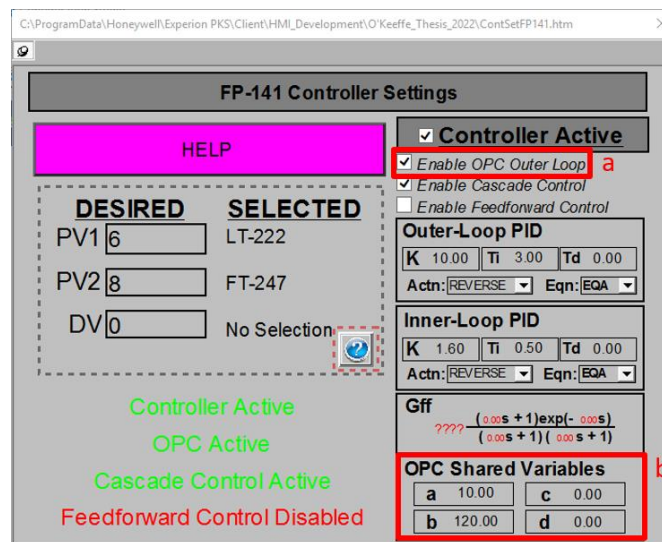


Figure 4: Feed Pump control popup, *a* shows OPC enable tick box, enabling this allows routing out of EPKS via OPC; *b* shows the OPC Shared Variables, these values can be read in an OPC compliant program, allowing configurable control parameters to be easily controlled from the EPKS HMI.

When OPC control is enabled, Cascade Control can still be employed using the enable cascade control tick box on the controller popup (Figure 4); when this box is unchecked, the output of "OPCControlTemplateR0" will act as the manipulated variable of the physical system. Alternatively, when the enable cascade control tick box is checked, the output of "OPCControlTemplateR0" will serve as the set point for the inner-loop cascaded controller in EPKS. **When OPC control is enabled, the 'Enable Feedforward' Control tick box has no effect. In the default configuration, the PV1 selected from the**

controller popup is routed to the OPC control program, and the controller setpoint is controlled from the HMI.

3.3 Modification of Program Control Schemes

In the previous section, it was stated that in its default state, the program is configured to enable PI control by each of the twelve controllers configured on EPKS. PI control is achieved via the velocity form, digital PI controller, given by:

$$mv_k = mv_{k-1} + K_c \left[\left(1 + \frac{\Delta t}{\tau_I} \right) \varepsilon_k - \varepsilon_{k-1} \right]$$

Figure 5 illustrates the implementation of this control scheme in the program for one of the controllers, namely FP-141; in the figure, we can see:

1. The n^{th} read of variables associated with controller 1 from the OPC server.
2. The case structure responsible for enabling controller function (*This case structure is checking OPC is enabled in both EPKS and the MATLAB program*).
3. The digital controller where:
 - a. OPC shared variables a and b are written to controller variables K_c and τ_I , respectively.
 - b. The error and time step n are calculated.
 - c. The control action at time step n is calculated.
 - d. A case structure is used to implement integral anti-windup before the desired mv 'tempmvDes' is written to the manipulated output variable 'mvDes(n)'.

```

cont1.fp141(n) = cell2mat(tempreadMvOut(2));
cont1.pv1(n) = cell2mat(tempreadPv1(2));
cont1.pv2(n) = cell2mat(tempreadPv2(2));
cont1.dv(n) = cell2mat(tempreadDv(2));
cont1.sp(n) = cell2mat(tempreadSp(2));
cont1.OPC(n) = cell2mat(tempOpcState(2));
cont1.OpcVarA = cell2mat(tempOpcVarA(2));
cont1.OpcVarB = cell2mat(tempOpcVarB(2));
cont1.OpcVarC = cell2mat(tempOpcVarC(2));
cont1.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont1.OPC(n) == 1 && cont1.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    % *** See Line 133

    % ----- FP141 Controller Goes Here -----
    % =====
    cont1.K = cont1.OpcVarA;
    cont1.ti = cont1.OpcVarB;

    cont1.err(n) = cont1.sp(n) - cont1.pv1(n);
    tempmvDes = cont1.mvDes(n-1) + cont1.K*((1+(dt/cont1.ti))*...
        *cont1.err(n) - cont1.err(n-1));

    if tempmvDes > 105
        cont1.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont1.mvDes(n) = -5;
    else
        cont1.mvDes(n) = tempmvDes;
    end

    % =====
    % =====
    % -----

```

Figure 5: Lines 671 to 705 of the 'OPCControlTemplateR0' code. The figure shows first the n^{th} read of controller 1 variables from the OPC server, then the configurable section of the feed pump controller set up to act as a simple discrete PI controller. OPC-shared variables A and B are imported from EPKS to act as controller gain and integrator time.

In most cases, the only modification required to change the control structure of a particular controller is the modification of the third itemized step listed above. Observing the variable reads in Figure 5, we can see that:

- the setpoint,
- process variables one and two,
- the disturbance variable,

Are all read from the OPC variables and are configured from the EPKS HMI. Thus, for cases when the variables required for the implementation of a controller can be found or routed to this set, the only program modification required lies in the area surrounded by green in Figure 5.

In cases where the variables already routed to the controller are insufficient for the desired control scheme, modifications to sections 2b, 3b, and 3c are also required. In these instances, modifications to the sections are as follows:

Section 2b:

Section 2b is intended to be used by users to create additional OPC communication groups if the desired control scheme requires more variables from EPKS than the number of variables already pre-routed in the program's default configuration. In the following descriptions of necessary changes for additional variable routing, we will use an example to better explain the procedure. Let us say for our desired control scheme; we need to additionally acquire tracking of the flow transmitter located at the outlet of the feed pump (FT-148). The EPKS control module in which this information exists has the point ID "FT-148"; thus, in section 2b of the program, we need to set up an OPC communication group to communicate with this CM; this is achieved with the following command:

```
ft148Group = addgroup(server, 'FT_148');
```

Here, 'Server' is a variable created at the beginning of the program associated with the MUPP OPC server, and 'FT_148' is the desired point ID. This procedure will need to be followed for any unique control module point IDs associated with additional desired MUPP parameters (*for an itemized list of MUPP point IDs and Parameters, see document "PP-1001: OPC Tag Names & Communication"*).

Section 3b:

Section 3b is intended to be used by users to create the point parameter reads associated with the OPC communication groups created in section 2b; on top of this, section 3b also houses the first read of the newly created variable. Following the example stated previously, the point parameter of the flow transmitter is 'FT_148.PV', and the command for creating the new point parameter is given by:

```
ft148Read = additem(ft148Group, 'FT_148.FT_148.PV');
```

Here, 'ft148Group' is the OPC Communication group defined in section 2b, and 'FT_148.FT_148.PV' is the point ID and point parameter of the FT-148 variable. Now that the point parameter has been defined, the

first read of the point parameter can be carried out. The output of a point parameter is a 1x5 matrix containing several additional pieces of information on top of the value of the variable in question. As a result, a temporary variable is required so that this matrix can first be saved to the temporary variable before the actual variable can assume the value of cell 2 in the matrix, which is the value of the read point parameter. These steps are given by:

```
temporaryVariable = struct2cell(read(ft148Read));
ft148(n) = cell2mat(temporaryVariable(2));
```

Section 3c:

Section 3c is intended to act as the continuously looped reader of all additional variables added by the program user in section 2b; as such, section 3c needs to house a copy of the same code utilized for the first read of each user-added variable, i.e.:

```
temporaryVariable = struct2cell(read(ft148Read));
ft148(n) = cell2mat(temporaryVariable(2));
```

Once any required variables have been added, construct the new control scheme in the controller section of the appropriate controller (*Figure 6*).

```
% ----- FP141 Controller Goes Here -----
% #####
% #####
cont1.K = cont1.OpcVarA;
cont1.ti = cont1.OpcVarB;

cont1.err(n) = cont1.sp(n) - cont1.pv1(n);
tempmvDes = cont1.mvDes(n-1) + cont1.K*((1+(dt/cont1.ti))*...
    *cont1.err(n) - cont1.err(n-1));
if tempmvDes > 105
    cont1.mvDes(n) = 105;
elseif tempmvDes < -5
    cont1.mvDes(n) = -5;
else
    cont1.mvDes(n) = tempmvDes;
end

% #####
% #####
% -----
```

Figure 6: Controller 1 controller section; intended for modification by program users.

4.0 Extensive Program Description

4.1 I. Initialization

1. Server Connect

The first section of the program is responsible for initiating a connection with the EPKS OPC DA server and setting the loop time of the main control loop, which will be started later (*Figure 7*). Since the MUPP OPC DA server details will not change, the only user-configurable parameter for this section is the loop time which, by default, is set to one second. During experimentation with the program, it was found that while all thirteen configured controllers are operating, the fastest achievable loop time is approximately 700 milliseconds. By commenting out unnecessary controllers, the loop time can be easily brought down to 500 milliseconds, the scan rate of the fastest-acting control elements in the MUPP. It is presumed that the limiting factor of a faster cycle rate is the hardware on either the server side 'PPServer1' side or the computer running the MATLAB program; however, further investigation would be required to say conclusively.

```
%%          1: Server Connect, Loop Time Set
server = opcda('ppserver1', 'HWHsc.OPCServer');
connect(server);

dt = 1;          % <---- Loop Time Set (Default = 1 second)
```

Figure 7: Section 1 of the 'OPCControlTemplateR0' MATLAB program.

2a. Control Group Establishment

Section 2a (*Figure 8*) serves the purpose of defining communication groups for each of the controllers configured in the program. The architecture of EPKS is such that communication groups are determined by the EPKS control builder control modules in the Honeywell EPKS controller. Since each EPKS controller is designed modularly so that all controller parameters exist in a single CM, only one communication group is required per controller.

```
%%          2a: Control Group Establishment
%          (Groups Synonymous With Control Modules
%          in EPKS)
cont1.grp = addgroup(server, 'FP_REF_141-R1A');    % Controller 1 CM
% Controller 2 group unestablished [BM 00C]
cont3.grp = addgroup(server, 'BMP_REF_241-R1A');    % Controller 3 CM
cont4.grp = addgroup(server, 'CRP_REF_341-R1A');    % Controller 4 CM
cont5.grp = addgroup(server, 'CUFP_REF_361-R1A');    % Controller 5 CM
% Controller 6 group unestablished
cont7.grp = addgroup(server, 'FDP_REF_521-R1A');    % Controller 7 CM
cont8.grp = addgroup(server, 'FCV_541-R1A');        % Controller 8 CM
cont9.grp = addgroup(server, 'NTP_REF_561-R1A');    % Controller 9 CM
% Controller 10 group unestablished [FCV-570 00C]
% Controller 11 group unestablished [FCV-574 00C]
% Controller 11 group unestablished [FCV-622 00C]
cont13.grp = addgroup(server, 'FCV_642-R1A');        % Controller 13 CM
cont14.grp = addgroup(server, 'FCV_662-R1A');        % Controller 14 CM
cont15.grp = addgroup(server, 'FCV_688-R1A');        % Controller 15 CM
cont16.grp = addgroup(server, 'FCV_690-R1A');        % Controller 16 CM
cont17.grp = addgroup(server, 'PP_REF_681-R1A');    % Controller 17 CM
% (18) SPARE CONTROLLER
% Controller 19 group unestablished [FCV-571 00C]
% (20) SPARE CONTROLLER
```

Figure 8: Section 2a of the 'OPCControlTemplateR0' MATLAB program.

2b. Bonus Variable Group Establishment

Section 2b (Figure 9), as explained in the previous section, is intended exclusively for program users as a location to define additional control groups if they are required. See '3.3 Modification of Program Control Schemes' for further explanation.

```
%%      2b: Bonus Variable Group Establishment
%      Use this field to add additional point parameter groups for
%      use in designed controllers, e.g.,
%      Bonus.ft148grp = addgroup(server, 'FT_148');
%      \_ > Flow Transmitter 148 Control Module
%
```

Figure 9: Section 2b of the 'OPCControlTemplateR0' MATLAB program.

3a. Variable Creation, Initialization, and Server Linking

Section 3a is responsible for initialising the iteration counter, n and the time array, t . Section 3a also defines all the point parameters required by the preconfigured controllers. Figure 10 illustrates only the definition and first read of point parameters related to controller one; however, the only differentiation between the definition of point parameters for controller one and the remaining controllers is the variable names and point-parameter identification.

The section starts by setting the iteration counter (n) to one and the first entry of the time array to zero; it then uses the inbuilt function *tic* to start a timer, the purpose of which will be explained in a later section. At this point, the point parameters for controller one can be defined. To begin, it is worth mentioning that controllers in this program are defined as objects, each variable associated with the controller is assigned to the controller object as a parameter of that object; this is done with the syntax '*object.parameter*', which can be observed throughout the program. This object-oriented program is carried out to make navigating the workspace easier as the program utilizes 200+ unique variables.

Point parameter definition is done in three blocks, the variables to be read, be variables that are both read from and written to, and the variables which are exclusively written to the OPC server. In all cases, the add item function is used, which first defines the communication group the point parameter belongs to (*defined in section 2a*), followed by the point parameter identification (*of the form PointID.PointParameter*), followed by the data format of the parameter.

As explained in 3.3 *Modification of Program Control Schemes*, the output of an OPC read is a 1x5 matrix; thus, a temporary variable must be utilized in which the read is first stored before being moved to the actual variable to be used for permanent storage. In section 3a, this is done in two separate blocks, where first, all variables of the associated controller are read to temporary variables. Then the applicable value from the read is stored in the appropriate variable. Once the first read of all the variables is carried out, three additional variables used only inside the MATLAB program are initialized. These are desired MV output, error, and previous loop OPC enable tracking.

It is worth reiterating that although this process is repeated for all configured controllers, the process is identical, with only the object, some variable names, and the Point ID and Point parameter details changing.


```

%%          3a: Variable Creation and Server Linking
% Initialization of n and t
n = 1;
t(n) = 0*dt;

tic % start timer

% Controller 1 Variable Creation
% Reads
cont1.fp141_read = additem(cont1.grp, 'FP_REF_141-R1A.FP_141.PV', 'double');
cont1.pv1_read = additem(cont1.grp, 'FP_REF_141-R1A.PV1.PV', 'double');
cont1.pv2_read = additem(cont1.grp, 'FP_REF_141-R1A.PV2.PV', 'double');
cont1.dv_read = additem(cont1.grp, 'FP_REF_141-R1A.DV.PV', 'double');
cont1.sp_read = additem(cont1.grp, 'FP_REF_141-R1A.SP.PV', 'double');
cont1.OpcVarA_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarA.PV', 'double');
cont1.OpcVarB_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarB.PV', 'double');
cont1.OpcVarC_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarC.PV', 'double');
cont1.OpcVarD_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont1.opcEnable = additem(cont1.grp, 'FP_REF_141-R1A.OPCenable.PVFL', 'double');

% Writes
cont1.fp141_write = additem(cont1.grp, 'FP_REF_141-R1A.FP141_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont1.fp141_read));
tempreadPv1 = struct2cell(read(cont1.pv1_read));
tempreadPv2 = struct2cell(read(cont1.pv2_read));
tempreadDv = struct2cell(read(cont1.dv_read));
tempreadSp = struct2cell(read(cont1.sp_read));
tempOpcState = struct2cell(read(cont1.opcEnable));
tempOpcVarA = struct2cell(read(cont1.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont1.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont1.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont1.OpcVarD_read));

cont1.fp141(n) = cell2mat(tempreadMvOut(2));
cont1.pv1(n) = cell2mat(tempreadPv1(2));
cont1.pv2(n) = cell2mat(tempreadPv2(2));
cont1.dv(n) = cell2mat(tempreadDv(2));
cont1.sp(n) = cell2mat(tempreadSp(2));
cont1.OPC(n) = cell2mat(tempOpcState(2));
cont1.OpcVarA = cell2mat(tempOpcVarA(2));
cont1.OpcVarB = cell2mat(tempOpcVarB(2));
cont1.OpcVarC = cell2mat(tempOpcVarC(2));
cont1.OpcVarD = cell2mat(tempOpcVarD(2));
cont1.mvDes(n) = 0;
cont1.err(n) = 0;
cont1.OPCprev = cont1.OPC(1);

```

Figure 10: An excerpt of section 3a of the 'OPCControlTemplateR0' program. The figure includes the initialization of program variables n and t, the start of the first program timer, and the creation and initialization of controller one variables.

3b. Bonus Variables Creation and First Read

Section 3b (Figure 11), as explained in the previous section, is intended exclusively for program users as a location to define additional point parameters. See '3.3 Modification of Program Control Schemes' for further explanation.

```

%%          3b: Bonus Variables Creation and First Read
% Use this field to create additional variables for use in
% controllers, for example to add variables for flow
% transmitter 148, noting bonus.ft148grp variable established
% in section 2b:
%
% bonus.ft148read = additem(bonus.ft148grp, 'FT_148.FT_148.PV', 'double');
% tempreadft148 = struct2cell(read(bonus.ft148read)); \
% bonus.ft148(n) = cell2mat(tempreadft148(2)); / <-- first read

```

Figure 11: Section 3b of the 'OPCControlTemplateR0' MATLAB program.

4.2 II. Main Loop (Controllers)

We now arrive at the initialization of the main loop, which is implemented as an infinite loop; thus, to end the program, it must be stopped. The first task in the main loop relates to timekeeping; earlier in the program, the *tic* function was used; we will now use the *toc* function to return the time elapsed. Briefly ignoring the conditional statement, we see that immediately following the statement, the timer is reset using the *tic* statement. Returning to the conditional statement, if the amount of time that has passed since the last *tic* function has run is less than the loop time of the program, the conditional statement will cause the program to wait until the loop time has elapsed. Thus, the loop time will be maintained; otherwise, the time that has passed will be displayed, notifying the user that the loop time is not being maintained. Finally, the iteration counter and time array are progressed before the controllers are run.

```
%%      II. Main Loop (Controllers)
while true
    timePassed = toc;           % \
    if timePassed < dt         % | If time since last timer reading
        pause(dt - timePassed) % | less than dt, wait sufficient time
    else                       % | to make sample time = dt, else
        disp(timePassed)      % | alert user
    end                       % /
    tic % reset timer

    n = n + 1; t(n) = (n-1)*dt; % progress n & t
```

Figure 12: Program main loop, initialization and housekeeping.

3c. Bonus Variables n^{th} Read

Section 3c (Figure 13), as explained in the previous section, is intended exclusively for program users as a location to carry out the n^{th} read of any point parameters defined in section 3b. See ‘3.3 Modification of Program Control Schemes’ for further explanation.

```
%%      3c: Bonus Variables Read
%      Use this field to read variables created in section 3b
%      tempreadft148 = struct2cell(read(bonus.ft148read)); \
%      bonus.ft148(n) = cell2mat(tempreadft148(2));      / <-- nth read
```

Figure 13: Section 3c of the ‘OPCControlTemplateR0’ MATLAB program.

4-23: Controllers 1-20

Much like the explanation of section 3a of the program, Figure 14 shows only section 4 of the code. Exploration of sections 5 to 23 is unnecessary as the structure and function of these sections are the same, save the factor that these sections apply to controllers 2-20. The first step in each controller is the MATLAB OPC enable variable (*MIOpcEnable*), which by default is set to false; the function of this variable will be elaborated on later. However, its purpose is to block or enable control OPC control on the MATLAB side of communication.

The next step of the code retrieves the n^{th} read of variables relevant to the controller from the OPC server. The process of conducting the n^{th} read is identical to performing the first read, so there is no need to elaborate on it further here. Following the n^{th} read, the program enters a conditional statement structure:

1. The first condition applies to cases where OPC is enabled on both the EPKS and MATLAB sides of OPC communication; in this case, the default PI controller or user-programmed controller runs.

2. The second condition applies to cases where OPC is enabled on the EPKS side of OPC communication but not the MATLAB side. In these cases, OPC is disabled on the EPKS side over OPC. Additionally, controller error for the time step is set to zero and desired MV on the MATLAB side is set to track the system's real MV to enable the bumpless transfer of control between EPKS and MATLAB control architectures.
3. The final condition applies to cases where OPC is disabled on both EPKS and MATLAB sides of OPC communication. For this condition, the controller error for the time step is set to zero and desired MV on the MATLAB side is set to track the real MV, much like in the previous case.

Following the case structure, the final steps of the section involve writing the desired MV value to EPKS via OPC and tracking the status of OPC from this time step for the next time step. The second task is helpful for control strategies where we would desire some unique task/s to happen for the first time in which OPC control is enabled or disabled.

```

%%      4: Controller 1, [FP-141 (BMT input pump)]
cont1.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%   MATLAB, else 0 will be written to OPC server.

%   updating controller Variables
tempreadMvOut = struct2cell(read(cont1.fp141_read));
tempreadPv1 = struct2cell(read(cont1.pv1_read));
tempreadPv2 = struct2cell(read(cont1.pv2_read));
tempreadDv = struct2cell(read(cont1.dv_read));
tempreadSp = struct2cell(read(cont1.sp_read));
tempOpcState = struct2cell(read(cont1.opcEnable));
tempOpcVarA = struct2cell(read(cont1.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont1.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont1.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont1.OpcVarD_read));

cont1.fp141(n) = cell2mat(tempreadMvOut(2));
cont1.pv1(n) = cell2mat(tempreadPv1(2));
cont1.pv2(n) = cell2mat(tempreadPv2(2));
cont1.dv(n) = cell2mat(tempreadDv(2));
cont1.sp(n) = cell2mat(tempreadSp(2));
cont1.OPC(n) = cell2mat(tempOpcState(2));
cont1.OpcVarA = cell2mat(tempOpcVarA(2));
cont1.OpcVarB = cell2mat(tempOpcVarB(2));
cont1.OpcVarC = cell2mat(tempOpcVarC(2));
cont1.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont1.OPC(n) == 1 && cont1.MlOpcEnable == 1)
%   ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
%   *** See Line 133

% ----- FP141 Controller Goes Here -----
% #####
% #####
cont1.K = cont1.OpcVarA;
cont1.ti = cont1.OpcVarB;

cont1.err(n) = cont1.sp(n) - cont1.pv1(n);
tempmvDes = cont1.mvDes(n-1) + cont1.K*((1+(dt/cont1.ti))...
    *cont1.err(n) - cont1.err(n-1));
if tempmvDes > 105
    cont1.mvDes(n) = 105;
elseif tempmvDes < -5
    cont1.mvDes(n) = -5;
else
    cont1.mvDes(n) = tempmvDes;
end

% #####
% #####
% -----
elseif (cont1.OPC(n) == 1 && cont1.MlOpcEnable == 0)
%   ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
%   disable OPC in EPKS

write(cont1.opcEnable, 0); % < Disable OPC in EPKS
cont1.err(n) = 0;
cont1.mvDes(n) = cont1.fp141(n);
else
%   \/\ If OPC disabled in EPKS and ML make error zero and make
%   MATLAB desired MV track EPKS outerloop MV.

cont1.err(n) = 0;
cont1.mvDes(n) = cont1.fp141(n);
end

% Write MV to EPKS
write(cont1.fp141_write, cont1.mvDes(n));
cont1.OPCprev = cont1.OPC(n);

```

Figure 14: Section 4 of the 'OPCControlTemplateR0' MATLAB program.

5.0 Further Reading

Consult Table 3 for recommendations for further reading.

Table 3: Recommended further reading and their relevance in the context of this document.

<i>Document:</i>	<i>Relevance:</i>
PP-1001: OPC Tag Names & Communication Rev.1	Useful for uncovering Point-ID and Point Parameter information for the variables available on the MUPP OPC server.
PP-5002: Operating the Pilot Plant via Station Rev.1	Useful for additional information on enabling OPC control from the EPKS HMI.
PP-5012: Standards of MUPP Sampling Time	This document outlines the limitations in sampling time on the EPKS server and gives insight into the practical limitations of the loop time of 'OPCControlTemplateR0'.

6.0 'OPCControlTemplateR0' – Code

```
%
%                               Pilot Plant OPC Control Template
%                               Nicholas O'Keeffe
%                               ICE, ICSE Thesis 2022
clear, close all, hold off
%
%                               '||'|' '||'|' '||'|' '||'|' '||'|'
%                               ||'|' '||'|' '||'|' '||'|' '||'|'
%                               ||'|' '||'|' '||'|' '||'|' '||'|'
%                               ||'|' '||'|' '||'|' '||'|' '||'|'
%                               ||'|' '||'|' '||'|' '||'|' '||'|'
%                               ||'|' '||'|' '||'|' '||'|' '||'|'
%
% Before using this program, it is strongly recommended that the
% Associated document 'PP-5007: 'OPCControlTemplateR0': Use and Function'
% is read. This document gives both a quick start guide to this program
% and a more extensive explanation of its function
%
% Some tips:
% - Navigate the program using the Go To menu on the editor ribbon of
%   the MATLAB toolbar
% - Once the program is known to be configured:
%   > Start the program using the run button.
%   > The program runs as an infinite loop so stop the program
%     using the stop button or ctrl+c
% - Export data using either the trendline feature of the EPKS HMI
%   OR, using the save() function from the MATLAB command line once
%   the program has been stopped.
%
%% Contents
% Sections:
% I. Initialization
% 1: Server Connect, Loop Time Set
% 2a: Control Group Establishment
% 2b: Bonus Variable Group Establishment
% 3a: Variable Creation, Initialization and Server Linking
% 3b: Bonus Variables Creation and First Read
% II. Main Loop (Controllers)
% 3c: Bonus Variables Read
% 4: Controller 1, [FP-141 (BMT input pump)]
% 5: Controller 2, [BM-201 (Ball Mill Speed)]
%   \__> *Out of Commission (OOC)*
% 6: Controller 3, [BMP-241 (BMT output pump)]
% 7: Controller 4, [CRP-341 (Cyclone Recycle Pump)]
% 8: Controller 5, [CUFP-361 (Cyclone Underflow Pump)]
% 9: Controller 6, [LUFP-421 (Lamella Underflow Pump)]
% 10: Controller 7, [FDP-521 (Feed Disturbance Pump)]
% 11: Controller 8, [FCV-541 (NLT input)]
% 12: Controller 9, [NTP-561 (Needle Tank Underflow Pump)]
% 13: Controller 10, [FCV-570 (CSTR1 input)]
% 14: Controller 11, [FCV-574 (NT to BMT)]-----*OOC*
% 15: Controller 12, [FCV-622 (CSTR1 Steam In)]-----*OOC*
% 16: Controller 13, [FCV-642 (CSTR2 Steam In)]
% 17: Controller 14, [FCV-662 (CSTR3 Steam In)]
% 18: Controller 15, [FCV-688 (System Output)]
% 19: Controller 16, [FCV-690 (CSTR3 Reflux)]
% 20: Controller 17, [PP-681 (Product Pump)]
% 21: Controller 18, [SPARE]
% 22: Controller 19, [FCV-571 (NT to ST Bank)]-----*OOC*
% 23: Controller 20, [SPARE]
%
%% I. Initialization
%% 1: Server Connect, Loop Time Set
server = opcda('ppserver1', 'HWHsc.OPCServer');
connect(server);

dt = 1; % <---- Loop Time Set (Default = 1 second)

%% 2a: Control Group Establishment
% (Groups Synonymous With Control Modules
% in EPKS)
cont1.grp = addgroup(server, 'FP_REF_141-R1A'); % Controller 1 CM
% Controller 2 group unestablished [BM OOC]
cont3.grp = addgroup(server, 'BMP_REF_241-R1A'); % Controller 3 CM
cont4.grp = addgroup(server, 'CRP_REF_341-R1A'); % Controller 4 CM
cont5.grp = addgroup(server, 'CUFP_REF_361-R1A'); % Controller 5 CM
% Controller 6 group unestablished
cont7.grp = addgroup(server, 'FDP_REF_521-R1A'); % Controller 7 CM
cont8.grp = addgroup(server, 'FCV_541-R1A'); % Controller 8 CM
```

```

cont9.grp = addgroup(server, 'NTP_REF_561-R1A'); % Controller 9 CM
cont10.grp = addgroup(server, 'FCV_570-R1A'); % Controller 10 CM
% Controller 11 group unestablished [FCV-574 OOC]
% Controller 12 group unestablished [FCV-622 OOC]
cont13.grp = addgroup(server, 'FCV_642-R1A'); % Controller 13 CM
cont14.grp = addgroup(server, 'FCV_662-R1A'); % Controller 14 CM
cont15.grp = addgroup(server, 'FCV_688-R1A'); % Controller 15 CM
cont16.grp = addgroup(server, 'FCV_690-R1A'); % Controller 16 CM
cont17.grp = addgroup(server, 'PP_REF_681-R1A'); % Controller 17 CM
% (18) SPARE CONTROLLER
% Controller 19 group unestablished [FCV-571 OOC]
% (20) SPARE CONTROLLER

%%      2b: Bonus Variable Group Establishment
%      Use this field to add additional point parameter groups for
%      use in designed controllers, e.g.,
%      Bonus.ft148grp = addgroup(server, 'FT_148');
%      \__> Flow Transmitter 148 Control Module

%%      3a: Variable Creation and Server Linking
% Initialization of n and t
n = 1;
t(n) = 0*dt;

tic % start timer

% Controller 1 Variable Creation
% Reads
cont1.fp141_read = additem(cont1.grp, 'FP_REF_141-R1A.FP_141.PV', 'double');
cont1.pv1_read = additem(cont1.grp, 'FP_REF_141-R1A.PV1.PV', 'double');
cont1.pv2_read = additem(cont1.grp, 'FP_REF_141-R1A.PV2.PV', 'double');
cont1.dv_read = additem(cont1.grp, 'FP_REF_141-R1A.DV.PV', 'double');
cont1.sp_read = additem(cont1.grp, 'FP_REF_141-R1A.SP.PV', 'double');
cont1.OpcVarA_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarA.PV', 'double');
cont1.OpcVarB_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarB.PV', 'double');
cont1.OpcVarC_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarC.PV', 'double');
cont1.OpcVarD_read = additem(cont1.grp, 'FP_REF_141-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont1.opcEnable = additem(cont1.grp, 'FP_REF_141-R1A.OPCenable.PVFL', 'double');

% Writes
cont1.fp141_write = additem(cont1.grp, 'FP_REF_141-R1A.FP141_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont1.fp141_read));
tempreadPv1 = struct2cell(read(cont1.pv1_read));
tempreadPv2 = struct2cell(read(cont1.pv2_read));
tempreadDv = struct2cell(read(cont1.dv_read));
tempreadSp = struct2cell(read(cont1.sp_read));
tempOpcState = struct2cell(read(cont1.opcEnable));
tempOpcVarA = struct2cell(read(cont1.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont1.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont1.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont1.OpcVarD_read));

cont1.fp141(n) = cell2mat(tempreadMvOut(2));
cont1.pv1(n) = cell2mat(tempreadPv1(2));
cont1.pv2(n) = cell2mat(tempreadPv2(2));
cont1.dv(n) = cell2mat(tempreadDv(2));
cont1.sp(n) = cell2mat(tempreadSp(2));
cont1.OPC(n) = cell2mat(tempOpcState(2));
cont1.OpcVarA = cell2mat(tempOpcVarA(2));
cont1.OpcVarB = cell2mat(tempOpcVarB(2));
cont1.OpcVarC = cell2mat(tempOpcVarC(2));
cont1.OpcVarD = cell2mat(tempOpcVarD(2));
cont1.mvDes(n) = 0;
cont1.err(n) = 0;
cont1.OPCprev = cont1.OPC(1);

% Controller 2 Variable Creation
% Controller 2 group unestablished [BM OOC]

% Controller 3 Variable Creation
% Reads
cont3.bmp241_read = additem(cont3.grp, 'BMP_REF_241-R1A.BMP_241.PV', 'double');
cont3.pv1_read = additem(cont3.grp, 'BMP_REF_241-R1A.PV1.PV', 'double');
cont3.pv2_read = additem(cont3.grp, 'BMP_REF_241-R1A.PV2.PV', 'double');

```

```

cont3.dv_read = additem(cont3.grp, 'BMP_REF_241-R1A.DV.PV', 'double');
cont3.sp_read = additem(cont3.grp, 'BMP_REF_241-R1A.SP.PV', 'double');
cont3.OpcVarA_read = additem(cont3.grp, 'BMP_REF_241-R1A.OPC_VarA.PV', 'double');
cont3.OpcVarB_read = additem(cont3.grp, 'BMP_REF_241-R1A.OPC_VarB.PV', 'double');
cont3.OpcVarC_read = additem(cont3.grp, 'BMP_REF_241-R1A.OPC_VarC.PV', 'double');
cont3.OpcVarD_read = additem(cont3.grp, 'BMP_REF_241-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont3.opcEnable = additem(cont3.grp, 'BMP_REF_241-R1A.OPCenable.PVFL', 'double');

% Writes
cont3.bmp241_write = additem(cont3.grp, 'BMP_REF_241-R1A.BMP241_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont3.bmp241_read));
tempreadPv1 = struct2cell(read(cont3.pv1_read));
tempreadPv2 = struct2cell(read(cont3.pv2_read));
tempreadDv = struct2cell(read(cont3.dv_read));
tempreadSp = struct2cell(read(cont3.sp_read));
tempOpcState = struct2cell(read(cont3.opcEnable));
tempOpcVarA = struct2cell(read(cont3.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont3.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont3.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont3.OpcVarD_read));

cont3.fpi141(n) = cell2mat(tempreadMvOut(2));
cont3.pv1(n) = cell2mat(tempreadPv1(2));
cont3.pv2(n) = cell2mat(tempreadPv2(2));
cont3.dv(n) = cell2mat(tempreadDv(2));
cont3.sp(n) = cell2mat(tempreadSp(2));
cont3.OPC(n) = cell2mat(tempOpcState(2));
cont3.OpcVarA = cell2mat(tempOpcVarA(2));
cont3.OpcVarB = cell2mat(tempOpcVarB(2));
cont3.OpcVarC = cell2mat(tempOpcVarC(2));
cont3.OpcVarD = cell2mat(tempOpcVarD(2));
cont3.mvDes(n) = 0;
cont3.err(n) = 0;
cont3.OPCprev = cont3.OPC(1);

% Controller 4 Variable Creation
% Reads
cont4.crp341_read = additem(cont4.grp, 'CRP_REF_341-R1A.CRP_341.PV', 'double');
cont4.pv1_read = additem(cont4.grp, 'CRP_REF_341-R1A.PV1.PV', 'double');
cont4.pv2_read = additem(cont4.grp, 'CRP_REF_341-R1A.PV2.PV', 'double');
cont4.dv_read = additem(cont4.grp, 'CRP_REF_341-R1A.DV.PV', 'double');
cont4.sp_read = additem(cont4.grp, 'CRP_REF_341-R1A.SP.PV', 'double');
cont4.OpcVarA_read = additem(cont4.grp, 'CRP_REF_341-R1A.OPC_VarA.PV', 'double');
cont4.OpcVarB_read = additem(cont4.grp, 'CRP_REF_341-R1A.OPC_VarB.PV', 'double');
cont4.OpcVarC_read = additem(cont4.grp, 'CRP_REF_341-R1A.OPC_VarC.PV', 'double');
cont4.OpcVarD_read = additem(cont4.grp, 'CRP_REF_341-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont4.opcEnable = additem(cont4.grp, 'CRP_REF_341-R1A.OPCenable.PVFL', 'double');

% Writes
cont4.crp341_write = additem(cont4.grp, 'CRP_REF_341-R1A.CRP341_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont4.crp341_read));
tempreadPv1 = struct2cell(read(cont4.pv1_read));
tempreadPv2 = struct2cell(read(cont4.pv2_read));
tempreadDv = struct2cell(read(cont4.dv_read));
tempreadSp = struct2cell(read(cont4.sp_read));
tempOpcState = struct2cell(read(cont4.opcEnable));
tempOpcVarA = struct2cell(read(cont4.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont4.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont4.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont4.OpcVarD_read));

cont4.crp341(n) = cell2mat(tempreadMvOut(2));
cont4.pv1(n) = cell2mat(tempreadPv1(2));
cont4.pv2(n) = cell2mat(tempreadPv2(2));
cont4.dv(n) = cell2mat(tempreadDv(2));
cont4.sp(n) = cell2mat(tempreadSp(2));
cont4.OPC(n) = cell2mat(tempOpcState(2));
cont4.OpcVarA = cell2mat(tempOpcVarA(2));

```



```

cont4.OpcVarB = cell2mat(tempOpcVarB(2));
cont4.OpcVarC = cell2mat(tempOpcVarC(2));
cont4.OpcVarD = cell2mat(tempOpcVarD(2));
cont4.mvDes(n) = 0;
cont4.err(n) = 0;
cont4.OPCprev = cont4.OPC(1);

% Controller 5 Variable Creation
% Reads
cont5.cufp361_read = additem(cont5.grp, 'CUFP_REF_361-R1A.CUFP_361.PV', 'double');
cont5.pv1_read = additem(cont5.grp, 'CUFP_REF_361-R1A.PV1.PV', 'double');
cont5.pv2_read = additem(cont5.grp, 'CUFP_REF_361-R1A.PV2.PV', 'double');
cont5.dv_read = additem(cont5.grp, 'CUFP_REF_361-R1A.DV.PV', 'double');
cont5.sp_read = additem(cont5.grp, 'CUFP_REF_361-R1A.SP.PV', 'double');
cont5.OpcVarA_read = additem(cont5.grp, 'CUFP_REF_361-R1A.OPC_VarA.PV', 'double');
cont5.OpcVarB_read = additem(cont5.grp, 'CUFP_REF_361-R1A.OPC_VarB.PV', 'double');
cont5.OpcVarC_read = additem(cont5.grp, 'CUFP_REF_361-R1A.OPC_VarC.PV', 'double');
cont5.OpcVarD_read = additem(cont5.grp, 'CUFP_REF_361-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont5.opcEnable = additem(cont5.grp, 'CUFP_REF_361-R1A.OPCenable.PVFL', 'double');

% Writes
cont5.cufp361_write = additem(cont5.grp, 'CUFP_REF_361-R1A.CUFP361_OPC.PV',
'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont5.cufp361_read));
tempreadPv1 = struct2cell(read(cont5.pv1_read));
tempreadPv2 = struct2cell(read(cont5.pv2_read));
tempreadDv = struct2cell(read(cont5.dv_read));
tempreadSp = struct2cell(read(cont5.sp_read));
tempOpcState = struct2cell(read(cont5.opcEnable));
tempOpcVarA = struct2cell(read(cont5.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont5.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont5.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont5.OpcVarD_read));

cont5.cufp361(n) = cell2mat(tempreadMvOut(2));
cont5.pv1(n) = cell2mat(tempreadPv1(2));
cont5.pv2(n) = cell2mat(tempreadPv2(2));
cont5.dv(n) = cell2mat(tempreadDv(2));
cont5.sp(n) = cell2mat(tempreadSp(2));
cont5.OPC(n) = cell2mat(tempOpcState(2));
cont5.OpcVarA = cell2mat(tempOpcVarA(2));
cont5.OpcVarB = cell2mat(tempOpcVarB(2));
cont5.OpcVarC = cell2mat(tempOpcVarC(2));
cont5.OpcVarD = cell2mat(tempOpcVarD(2));
cont5.mvDes(n) = 0;
cont5.err(n) = 0;
cont5.OPCprev = cont5.OPC(1);

% Controller 6 Variable Creation
% Controller 6 group unestablished -- LUP unused

% Controller 7 Variable Creation
% Reads
cont7.fdp521_read = additem(cont7.grp, 'FDP_REF_521-R1A.FDP_521.PV', 'double');
cont7.pv1_read = additem(cont7.grp, 'FDP_REF_521-R1A.PV1.PV', 'double');
cont7.pv2_read = additem(cont7.grp, 'FDP_REF_521-R1A.PV2.PV', 'double');
cont7.dv_read = additem(cont7.grp, 'FDP_REF_521-R1A.DV.PV', 'double');
cont7.sp_read = additem(cont7.grp, 'FDP_REF_521-R1A.SP.PV', 'double');
cont7.OpcVarA_read = additem(cont7.grp, 'FDP_REF_521-R1A.OPC_VarA.PV', 'double');
cont7.OpcVarB_read = additem(cont7.grp, 'FDP_REF_521-R1A.OPC_VarB.PV', 'double');
cont7.OpcVarC_read = additem(cont7.grp, 'FDP_REF_521-R1A.OPC_VarC.PV', 'double');
cont7.OpcVarD_read = additem(cont7.grp, 'FDP_REF_521-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont7.opcEnable = additem(cont7.grp, 'FDP_REF_521-R1A.OPCenable.PVFL', 'double');

% Writes
cont7.fdp521_write = additem(cont7.grp, 'FDP_REF_521-R1A.FDP521_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont7.fdp521_read));
tempreadPv1 = struct2cell(read(cont7.pv1_read));
tempreadPv2 = struct2cell(read(cont7.pv2_read));
tempreadDv = struct2cell(read(cont7.dv_read));

```

```

tempreadSp = struct2cell(read(cont7.sp_read));
tempOpcState = struct2cell(read(cont7.opcEnable));
tempOpcVarA = struct2cell(read(cont7.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont7.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont7.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont7.OpcVarD_read));

cont7.fdp521(n) = cell2mat(tempreadMvOut(2));
cont7.pv1(n) = cell2mat(tempreadPv1(2));
cont7.pv2(n) = cell2mat(tempreadPv2(2));
cont7.dv(n) = cell2mat(tempreadDv(2));
cont7.sp(n) = cell2mat(tempreadSp(2));
cont7.OPC(n) = cell2mat(tempOpcState(2));
cont7.OpcVarA = cell2mat(tempOpcVarA(2));
cont7.OpcVarB = cell2mat(tempOpcVarB(2));
cont7.OpcVarC = cell2mat(tempOpcVarC(2));
cont7.OpcVarD = cell2mat(tempOpcVarD(2));
cont7.mvDes(n) = 0;
cont7.err(n) = 0;
cont7.OPCprev = cont7.OPC(1);

% Controller 8 Variable Creation
% Reads
cont8.fcv541_read = additem(cont8.grp, 'FCV_541-R1A.FCV_541.PV', 'double');
cont8.pv1_read = additem(cont8.grp, 'FCV_541-R1A.PV1.PV', 'double');
cont8.pv2_read = additem(cont8.grp, 'FCV_541-R1A.PV2.PV', 'double');
cont8.dv_read = additem(cont8.grp, 'FCV_541-R1A.DV.PV', 'double');
cont8.sp_read = additem(cont8.grp, 'FCV_541-R1A.SP.PV', 'double');
cont8.OpcVarA_read = additem(cont8.grp, 'FCV_541-R1A.OPC_VarA.PV', 'double');
cont8.OpcVarB_read = additem(cont8.grp, 'FCV_541-R1A.OPC_VarB.PV', 'double');
cont8.OpcVarC_read = additem(cont8.grp, 'FCV_541-R1A.OPC_VarC.PV', 'double');
cont8.OpcVarD_read = additem(cont8.grp, 'FCV_541-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont8.opcEnable = additem(cont8.grp, 'FCV_541-R1A.OPCenable.PVFL', 'double');

% Writes
cont8.fcv541_write = additem(cont8.grp, 'FCV_541-R1A.FCV541_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont8.fcv541_read));
tempreadPv1 = struct2cell(read(cont8.pv1_read));
tempreadPv2 = struct2cell(read(cont8.pv2_read));
tempreadDv = struct2cell(read(cont8.dv_read));
tempreadSp = struct2cell(read(cont8.sp_read));
tempOpcState = struct2cell(read(cont8.opcEnable));
tempOpcVarA = struct2cell(read(cont8.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont8.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont8.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont8.OpcVarD_read));

cont8.fcv541(n) = cell2mat(tempreadMvOut(2));
cont8.pv1(n) = cell2mat(tempreadPv1(2));
cont8.pv2(n) = cell2mat(tempreadPv2(2));
cont8.dv(n) = cell2mat(tempreadDv(2));
cont8.sp(n) = cell2mat(tempreadSp(2));
cont8.OPC(n) = cell2mat(tempOpcState(2));
cont8.OpcVarA = cell2mat(tempOpcVarA(2));
cont8.OpcVarB = cell2mat(tempOpcVarB(2));
cont8.OpcVarC = cell2mat(tempOpcVarC(2));
cont8.OpcVarD = cell2mat(tempOpcVarD(2));
cont8.mvDes(n) = 0;
cont8.err(n) = 0;
cont8.OPCprev = cont8.OPC(1);

% Controller 9 Variable Creation
% Reads
cont9.ntp561_read = additem(cont9.grp, 'NTP_REF_561-R1A.NTP_561.PV', 'double');
cont9.pv1_read = additem(cont9.grp, 'NTP_REF_561-R1A.PV1.PV', 'double');
cont9.pv2_read = additem(cont9.grp, 'NTP_REF_561-R1A.PV2.PV', 'double');
cont9.dv_read = additem(cont9.grp, 'NTP_REF_561-R1A.DV.PV', 'double');
cont9.sp_read = additem(cont9.grp, 'NTP_REF_561-R1A.SP.PV', 'double');
cont9.OpcVarA_read = additem(cont9.grp, 'NTP_REF_561-R1A.OPC_VarA.PV', 'double');
cont9.OpcVarB_read = additem(cont9.grp, 'NTP_REF_561-R1A.OPC_VarB.PV', 'double');
cont9.OpcVarC_read = additem(cont9.grp, 'NTP_REF_561-R1A.OPC_VarC.PV', 'double');
cont9.OpcVarD_read = additem(cont9.grp, 'NTP_REF_561-R1A.OPC_VarD.PV', 'double');

% Read/Write

```

```

cont9.opcEnable = additem(cont9.grp, 'NTP_REF_561-R1A.OPCenable.PVFL', 'double');

% Writes
cont9.ntp561_write = additem(cont9.grp, 'NTP_REF_561-R1A.NTP561_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont9.ntp561_read));
tempreadPv1 = struct2cell(read(cont9.pv1_read));
tempreadPv2 = struct2cell(read(cont9.pv2_read));
tempreadDv = struct2cell(read(cont9.dv_read));
tempreadSp = struct2cell(read(cont9.sp_read));
tempOpcState = struct2cell(read(cont9.opcEnable));
tempOpcVarA = struct2cell(read(cont9.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont9.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont9.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont9.OpcVarD_read));

cont9.ntp561(n) = cell2mat(tempreadMvOut(2));
cont9.pv1(n) = cell2mat(tempreadPv1(2));
cont9.pv2(n) = cell2mat(tempreadPv2(2));
cont9.dv(n) = cell2mat(tempreadDv(2));
cont9.sp(n) = cell2mat(tempreadSp(2));
cont9.OPC(n) = cell2mat(tempOpcState(2));
cont9.OpcVarA = cell2mat(tempOpcVarA(2));
cont9.OpcVarB = cell2mat(tempOpcVarB(2));
cont9.OpcVarC = cell2mat(tempOpcVarC(2));
cont9.OpcVarD = cell2mat(tempOpcVarD(2));
cont9.mvDes(n) = 0;
cont9.err(n) = 0;
cont9.OPCprev = cont9.OPC(1);

% Controller 10 Variable Creation
% Reads
cont10.fcv570_read = additem(cont10.grp, 'FCV_570-R1A.FCV_570.PV', 'double');
cont10.pv1_read = additem(cont10.grp, 'FCV_570-R1A.PV1.PV', 'double');
cont10.pv2_read = additem(cont10.grp, 'FCV_570-R1A.PV2.PV', 'double');
cont10.dv_read = additem(cont10.grp, 'FCV_570-R1A.DV.PV', 'double');
cont10.sp_read = additem(cont10.grp, 'FCV_570-R1A.SP.PV', 'double');
cont10.OpcVarA_read = additem(cont10.grp, 'FCV_570-R1A.OPC_VarA.PV', 'double');
cont10.OpcVarB_read = additem(cont10.grp, 'FCV_570-R1A.OPC_VarB.PV', 'double');
cont10.OpcVarC_read = additem(cont10.grp, 'FCV_570-R1A.OPC_VarC.PV', 'double');
cont10.OpcVarD_read = additem(cont10.grp, 'FCV_570-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont10.opcEnable = additem(cont10.grp, 'FCV_570-R1A.OPCenable.PVFL', 'double');

% Writes
cont10.fcv570_write = additem(cont10.grp, 'FCV_570-R1A.FCV570_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont10.fcv570_read));
tempreadPv1 = struct2cell(read(cont10.pv1_read));
tempreadPv2 = struct2cell(read(cont10.pv2_read));
tempreadDv = struct2cell(read(cont10.dv_read));
tempreadSp = struct2cell(read(cont10.sp_read));
tempOpcState = struct2cell(read(cont10.opcEnable));
tempOpcVarA = struct2cell(read(cont10.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont10.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont10.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont10.OpcVarD_read));

cont10.fcv541(n) = cell2mat(tempreadMvOut(2));
cont10.pv1(n) = cell2mat(tempreadPv1(2));
cont10.pv2(n) = cell2mat(tempreadPv2(2));
cont10.dv(n) = cell2mat(tempreadDv(2));
cont10.sp(n) = cell2mat(tempreadSp(2));
cont10.OPC(n) = cell2mat(tempOpcState(2));
cont10.OpcVarA = cell2mat(tempOpcVarA(2));
cont10.OpcVarB = cell2mat(tempOpcVarB(2));
cont10.OpcVarC = cell2mat(tempOpcVarC(2));
cont10.OpcVarD = cell2mat(tempOpcVarD(2));
cont10.mvDes(n) = 0;
cont10.err(n) = 0;
cont10.OPCprev = cont10.OPC(1);

% Controller 11 Variable Creation
% Controller 11 group unestablished [FCV-574 OOC]

```

```

% Controller 12 Variable Creation
% Controller 12 group unestablished [FCV-622 OOC]

% Controller 13 Variable Creation
% Reads
cont13.fcv642_read = additem(cont13.grp, 'FCV_642-R1A.FCV_642.PV', 'double');
cont13.pv1_read = additem(cont13.grp, 'FCV_642-R1A.PV1.PV', 'double');
cont13.pv2_read = additem(cont13.grp, 'FCV_642-R1A.PV2.PV', 'double');
cont13.dv_read = additem(cont13.grp, 'FCV_642-R1A.DV.PV', 'double');
cont13.sp_read = additem(cont13.grp, 'FCV_642-R1A.SP.PV', 'double');
cont13.OpcVarA_read = additem(cont13.grp, 'FCV_642-R1A.OPC_VarA.PV', 'double');
cont13.OpcVarB_read = additem(cont13.grp, 'FCV_642-R1A.OPC_VarB.PV', 'double');
cont13.OpcVarC_read = additem(cont13.grp, 'FCV_642-R1A.OPC_VarC.PV', 'double');
cont13.OpcVarD_read = additem(cont13.grp, 'FCV_642-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont13.opcEnable = additem(cont13.grp, 'FCV_642-R1A.OPCenable.PVFL', 'double');

% Writes
cont13.fcv642_write = additem(cont13.grp, 'FCV_642-R1A.FCV642_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont13.fcv642_read));
tempreadPv1 = struct2cell(read(cont13.pv1_read));
tempreadPv2 = struct2cell(read(cont13.pv2_read));
tempreadDv = struct2cell(read(cont13.dv_read));
tempreadSp = struct2cell(read(cont13.sp_read));
tempOpcState = struct2cell(read(cont13.opcEnable));
tempOpcVarA = struct2cell(read(cont13.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont13.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont13.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont13.OpcVarD_read));

cont13.fcv642(n) = cell2mat(tempreadMvOut(2));
cont13.pv1(n) = cell2mat(tempreadPv1(2));
cont13.pv2(n) = cell2mat(tempreadPv2(2));
cont13.dv(n) = cell2mat(tempreadDv(2));
cont13.sp(n) = cell2mat(tempreadSp(2));
cont13.OPC(n) = cell2mat(tempOpcState(2));
cont13.OpcVarA = cell2mat(tempOpcVarA(2));
cont13.OpcVarB = cell2mat(tempOpcVarB(2));
cont13.OpcVarC = cell2mat(tempOpcVarC(2));
cont13.OpcVarD = cell2mat(tempOpcVarD(2));
cont13.mvDes(n) = 0;
cont13.err(n) = 0;
cont13.OPCprev = cont13.OPC(1);

% Controller 14 Variable Creation
% Reads
cont14.fcv662_read = additem(cont14.grp, 'FCV_662-R1A.FCV_662.PV', 'double');
cont14.pv1_read = additem(cont14.grp, 'FCV_662-R1A.PV1.PV', 'double');
cont14.pv2_read = additem(cont14.grp, 'FCV_662-R1A.PV2.PV', 'double');
cont14.dv_read = additem(cont14.grp, 'FCV_662-R1A.DV.PV', 'double');
cont14.sp_read = additem(cont14.grp, 'FCV_662-R1A.SP.PV', 'double');
cont14.OpcVarA_read = additem(cont14.grp, 'FCV_662-R1A.OPC_VarA.PV', 'double');
cont14.OpcVarB_read = additem(cont14.grp, 'FCV_662-R1A.OPC_VarB.PV', 'double');
cont14.OpcVarC_read = additem(cont14.grp, 'FCV_662-R1A.OPC_VarC.PV', 'double');
cont14.OpcVarD_read = additem(cont14.grp, 'FCV_662-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont14.opcEnable = additem(cont14.grp, 'FCV_662-R1A.OPCenable.PVFL', 'double');

% Writes
cont14.fcv662_write = additem(cont14.grp, 'FCV_662-R1A.FCV662_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont14.fcv662_read));
tempreadPv1 = struct2cell(read(cont14.pv1_read));
tempreadPv2 = struct2cell(read(cont14.pv2_read));
tempreadDv = struct2cell(read(cont14.dv_read));
tempreadSp = struct2cell(read(cont14.sp_read));
tempOpcState = struct2cell(read(cont14.opcEnable));
tempOpcVarA = struct2cell(read(cont14.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont14.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont14.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont14.OpcVarD_read));

cont14.fcv662(n) = cell2mat(tempreadMvOut(2));

```

```

cont14.pv1(n) = cell2mat(tempreadPv1(2));
cont14.pv2(n) = cell2mat(tempreadPv2(2));
cont14.dv(n) = cell2mat(tempreadDv(2));
cont14.sp(n) = cell2mat(tempreadSp(2));
cont14.OPC(n) = cell2mat(tempOpcState(2));
cont14.OpcVarA = cell2mat(tempOpcVarA(2));
cont14.OpcVarB = cell2mat(tempOpcVarB(2));
cont14.OpcVarC = cell2mat(tempOpcVarC(2));
cont14.OpcVarD = cell2mat(tempOpcVarD(2));
cont14.mvDes(n) = 0;
cont14.err(n) = 0;
cont14.OPCprev = cont14.OPC(1);

% Controller 15 Variable Creation
% Reads
cont15.fcv688_read = additem(cont14.grp, 'FCV_688-R1A.FCV_688.PV', 'double');
cont15.pv1_read = additem(cont14.grp, 'FCV_688-R1A.PV1.PV', 'double');
cont15.pv2_read = additem(cont14.grp, 'FCV_688-R1A.PV2.PV', 'double');
cont15.dv_read = additem(cont14.grp, 'FCV_688-R1A.DV.PV', 'double');
cont15.sp_read = additem(cont14.grp, 'FCV_688-R1A.SP.PV', 'double');
cont15.OpcVarA_read = additem(cont15.grp, 'FCV_688-R1A.OPC_VarA.PV', 'double');
cont15.OpcVarB_read = additem(cont15.grp, 'FCV_688-R1A.OPC_VarB.PV', 'double');
cont15.OpcVarC_read = additem(cont15.grp, 'FCV_688-R1A.OPC_VarC.PV', 'double');
cont15.OpcVarD_read = additem(cont15.grp, 'FCV_688-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont15.opcEnable = additem(cont14.grp, 'FCV_688-R1A.OPCenable.PVFL', 'double');

% Writes
cont15.fcv688_write = additem(cont14.grp, 'FCV_688-R1A.FCV688_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont15.fcv688_read));
tempreadPv1 = struct2cell(read(cont15.pv1_read));
tempreadPv2 = struct2cell(read(cont15.pv2_read));
tempreadDv = struct2cell(read(cont15.dv_read));
tempreadSp = struct2cell(read(cont15.sp_read));
tempOpcState = struct2cell(read(cont15.opcEnable));
tempOpcVarA = struct2cell(read(cont15.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont15.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont15.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont15.OpcVarD_read));

cont15.fcv688(n) = cell2mat(tempreadMvOut(2));
cont15.pv1(n) = cell2mat(tempreadPv1(2));
cont15.pv2(n) = cell2mat(tempreadPv2(2));
cont15.dv(n) = cell2mat(tempreadDv(2));
cont15.sp(n) = cell2mat(tempreadSp(2));
cont15.OPC(n) = cell2mat(tempOpcState(2));
cont15.OpcVarA = cell2mat(tempOpcVarA(2));
cont15.OpcVarB = cell2mat(tempOpcVarB(2));
cont15.OpcVarC = cell2mat(tempOpcVarC(2));
cont15.OpcVarD = cell2mat(tempOpcVarD(2));
cont15.mvDes(n) = 0;
cont15.err(n) = 0;
cont15.OPCprev = cont15.OPC(1);

% Controller 16 Variable Creation
% Reads
cont16.fcv690_read = additem(cont16.grp, 'FCV_690-R1A.FCV_690.PV', 'double');
cont16.pv1_read = additem(cont16.grp, 'FCV_690-R1A.PV1.PV', 'double');
cont16.pv2_read = additem(cont16.grp, 'FCV_690-R1A.PV2.PV', 'double');
cont16.dv_read = additem(cont16.grp, 'FCV_690-R1A.DV.PV', 'double');
cont16.sp_read = additem(cont16.grp, 'FCV_690-R1A.SP.PV', 'double');
cont16.OpcVarA_read = additem(cont16.grp, 'FCV_690-R1A.OPC_VarA.PV', 'double');
cont16.OpcVarB_read = additem(cont16.grp, 'FCV_690-R1A.OPC_VarB.PV', 'double');
cont16.OpcVarC_read = additem(cont16.grp, 'FCV_690-R1A.OPC_VarC.PV', 'double');
cont16.OpcVarD_read = additem(cont16.grp, 'FCV_690-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont16.opcEnable = additem(cont16.grp, 'FCV_690-R1A.OPCenable.PVFL', 'double');

% Writes
cont16.fcv690_write = additem(cont16.grp, 'FCV_690-R1A.FCV690_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont16.fcv690_read));
tempreadPv1 = struct2cell(read(cont16.pv1_read));

```

```

tempreadPv2 = struct2cell(read(cont16.pv2_read));
tempreadDv = struct2cell(read(cont16.dv_read));
tempreadSp = struct2cell(read(cont16.sp_read));
tempOpcState = struct2cell(read(cont16.opcEnable));
tempOpcVarA = struct2cell(read(cont16.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont16.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont16.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont16.OpcVarD_read));

cont16.fcv690(n) = cell2mat(tempreadMvOut(2));
cont16.pv1(n) = cell2mat(tempreadPv1(2));
cont16.pv2(n) = cell2mat(tempreadPv2(2));
cont16.dv(n) = cell2mat(tempreadDv(2));
cont16.sp(n) = cell2mat(tempreadSp(2));
cont16.OPC(n) = cell2mat(tempOpcState(2));
cont16.OpcVarA = cell2mat(tempOpcVarA(2));
cont16.OpcVarB = cell2mat(tempOpcVarB(2));
cont16.OpcVarC = cell2mat(tempOpcVarC(2));
cont16.OpcVarD = cell2mat(tempOpcVarD(2));
cont16.mvDes(n) = 0;
cont16.err(n) = 0;
cont16.OPCprev = cont16.OPC(1);

% Controller 17 Variable Creation
% Reads
cont17.pp681_read = additem(cont17.grp, 'PP_REF_681-R1A.PP_681.PV', 'double');
cont17.pv1_read = additem(cont17.grp, 'PP_REF_681-R1A.PV1.PV', 'double');
cont17.pv2_read = additem(cont17.grp, 'PP_REF_681-R1A.PV2.PV', 'double');
cont17.dv_read = additem(cont17.grp, 'PP_REF_681-R1A.DV.PV', 'double');
cont17.sp_read = additem(cont17.grp, 'PP_REF_681-R1A.SP.PV', 'double');
cont17.OpcVarA_read = additem(cont17.grp, 'PP_REF_681-R1A.OPC_VarA.PV', 'double');
cont17.OpcVarB_read = additem(cont17.grp, 'PP_REF_681-R1A.OPC_VarB.PV', 'double');
cont17.OpcVarC_read = additem(cont17.grp, 'PP_REF_681-R1A.OPC_VarC.PV', 'double');
cont17.OpcVarD_read = additem(cont17.grp, 'PP_REF_681-R1A.OPC_VarD.PV', 'double');

% Read/Write
cont17.opcEnable = additem(cont17.grp, 'PP_REF_681-R1A.OPCenable.PVFL', 'double');

% Writes
cont17.pp681_write = additem(cont17.grp, 'PP_REF_681-R1A.PP681_OPC.PV', 'double');

% initialize controller variables
tempreadMvOut = struct2cell(read(cont17.pp681_read));
tempreadPv1 = struct2cell(read(cont17.pv1_read));
tempreadPv2 = struct2cell(read(cont17.pv2_read));
tempreadDv = struct2cell(read(cont17.dv_read));
tempreadSp = struct2cell(read(cont17.sp_read));
tempOpcState = struct2cell(read(cont17.opcEnable));
tempOpcVarA = struct2cell(read(cont17.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont17.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont17.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont17.OpcVarD_read));

cont17.pp681(n) = cell2mat(tempreadMvOut(2));
cont17.pv1(n) = cell2mat(tempreadPv1(2));
cont17.pv2(n) = cell2mat(tempreadPv2(2));
cont17.dv(n) = cell2mat(tempreadDv(2));
cont17.sp(n) = cell2mat(tempreadSp(2));
cont17.OPC(n) = cell2mat(tempOpcState(2));
cont17.OpcVarA = cell2mat(tempOpcVarA(2));
cont17.OpcVarB = cell2mat(tempOpcVarB(2));
cont17.OpcVarC = cell2mat(tempOpcVarC(2));
cont17.OpcVarD = cell2mat(tempOpcVarD(2));
cont17.mvDes(n) = 0;
cont17.err(n) = 0;
cont17.OPCprev = cont17.OPC(1);

% Controller 18 Variable Creation
% (18) SPARE CONTROLLER

% Controller 19 Variable Creation
% Controller 19 group unestablished [FCV-571 OOC]

% Controller 20 Variable Creation
% (20) SPARE CONTROLLER

%%          3b: Bonus Variables Creation and First Read
%          Use this field to create additional variables for use in

```

```

%           controllers, for example to add variables for flow
%           transmitter 148, noting bonus.ft148grp variable established
%           in section 2b:
%
% bonus.ft148read = additem(bonus.ft148grp, 'FT_148.FT_148.PV', 'double');
% tempreadft148 = struct2cell(read(bonus.ft148read)); \
% bonus.ft148(n) = cell2mat(tempreadft148(2)); / <-- first read

%%           II. Main Loop (Controllers)
while true
    timePassed = toc; % \
    if timePassed < dt % | If time since last timer reading
        pause(dt - timePassed) % | less than dt, wait sufficient time
    else % | to make sample time = dt, else
        disp(timePassed) % | alert user
    end % /
    tic % reset timer

    n = n + 1; t(n) = (n-1)*dt; % progress n & t

    %%           3c: Bonus Variables Read
    %           Use this field to read variables created in section 3b
    % tempreadft148 = struct2cell(read(bonus.ft148read)); \
    % bonus.ft148(n) = cell2mat(tempreadft148(2)); / <-- nth read

    %%           4: Controller 1, [FP-141 (BMT input pump)]
    cont1.MlOpcEnable = false;
    % ^^^ This be made true to enable any controllers configured in
    % MATLAB, else 0 will be written to OPC server.

    % updating controller Variables
    tempreadMvOut = struct2cell(read(cont1.fp141_read));
    tempreadPv1 = struct2cell(read(cont1.pv1_read));
    tempreadPv2 = struct2cell(read(cont1.pv2_read));
    tempreadDv = struct2cell(read(cont1.dv_read));
    tempreadSp = struct2cell(read(cont1.sp_read));
    tempOpcState = struct2cell(read(cont1.opcEnable));
    tempOpcVarA = struct2cell(read(cont1.OpcVarA_read));
    tempOpcVarB = struct2cell(read(cont1.OpcVarB_read));
    tempOpcVarC = struct2cell(read(cont1.OpcVarC_read));
    tempOpcVarD = struct2cell(read(cont1.OpcVarD_read));

    cont1.fp141(n) = cell2mat(tempreadMvOut(2));
    cont1.pv1(n) = cell2mat(tempreadPv1(2));
    cont1.pv2(n) = cell2mat(tempreadPv2(2));
    cont1.dv(n) = cell2mat(tempreadDv(2));
    cont1.sp(n) = cell2mat(tempreadSp(2));
    cont1.OPC(n) = cell2mat(tempOpcState(2));
    cont1.OpcVarA = cell2mat(tempOpcVarA(2));
    cont1.OpcVarB = cell2mat(tempOpcVarB(2));
    cont1.OpcVarC = cell2mat(tempOpcVarC(2));
    cont1.OpcVarD = cell2mat(tempOpcVarD(2));

    if (cont1.OPC(n) == 1 && cont1.MlOpcEnable == 1)
        % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
        % *** See Line 133

        % ----- FP141 Controller Goes Here -----
        % #####
        % #####
        cont1.K = cont1.OpcVarA;
        cont1.ti = cont1.OpcVarB;

        cont1.err(n) = cont1.sp(n) - cont1.pv1(n);
        tempmvDes = cont1.mvDes(n-1) + cont1.K*((1+(dt/cont1.ti)).^n - 1) * cont1.err(n) - cont1.err(n-1);
        if tempmvDes > 105
            cont1.mvDes(n) = 105;
        elseif tempmvDes < -5
            cont1.mvDes(n) = -5;
        else
            cont1.mvDes(n) = tempmvDes;
        end

        % #####
        % #####
        % -----

```

```

elseif (cont1.OPC(n) == 1 && cont1.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont1.opcEnable, 0); % < Disable OPC in EPKS
    cont1.err(n) = 0;
    cont1.mvDes(n) = cont1.fp141(n);
else
    % \/\ / If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont1.err(n) = 0;
    cont1.mvDes(n) = cont1.fp141(n);
end

% Write MV to EPKS
write(cont1.fp141_write, cont1.mvDes(n));
cont1.OPCprev = cont1.OPC(n);

%%
%      5: Controller 2, [BM-201 (Ball Mill Speed)] *OOC*
% This asset is currently out of commission, leave empty until
% asset is operational.
%
%
%
%
%
%
%      6: Controller 3, [BMP-241 (BMT output pump)]
cont3.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%      MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont3.bmp241_read));
tempreadPv1 = struct2cell(read(cont3.pv1_read));
tempreadPv2 = struct2cell(read(cont3.pv2_read));
tempreadDv = struct2cell(read(cont3.dv_read));
tempreadSp = struct2cell(read(cont3.sp_read));
tempOpcState = struct2cell(read(cont3.opcEnable));
tempOpcVarA = struct2cell(read(cont3.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont3.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont3.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont3.OpcVarD_read));

cont3.bmp241(n) = cell2mat(tempreadMvOut(2));
cont3.pv1(n) = cell2mat(tempreadPv1(2));
cont3.pv2(n) = cell2mat(tempreadPv2(2));
cont3.dv(n) = cell2mat(tempreadDv(2));
cont3.sp(n) = cell2mat(tempreadSp(2));
cont3.OPC(n) = cell2mat(tempOpcState(2));
cont3.OpcVarA = cell2mat(tempOpcVarA(2));
cont3.OpcVarB = cell2mat(tempOpcVarB(2));
cont3.OpcVarC = cell2mat(tempOpcVarC(2));
cont3.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont3.OPC(n) == 1 && cont3.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %      *** See Line 133

    % ----- BMP241 Controller Goes Here -----
    % #####
    % #####
    cont3.K = cont3.OpcVarA;
    cont3.ti = cont3.OpcVarB;

    cont3.err(n) = cont3.sp(n) - cont3.pv1(n);
    tempmvDes = cont3.mvDes(n-1) + cont3.K*((1+(dt/cont3.ti))...
        *cont3.err(n) - cont3.err(n-1));
    if tempmvDes > 105
        cont3.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont3.mvDes(n) = -5;
    else
        cont3.mvDes(n) = tempmvDes;
    end

    % #####
    % #####

```



```

% -----
elseif (cont3.OPC(n) == 1 && cont3.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont3.opcEnable, 0); % < Disable OPC in EPKS
    cont3.err(n) = 0;
    cont3.mvDes(n) = cont3.bmp241(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont3.err(n) = 0;
    cont3.mvDes(n) = cont3.bmp241(n);
end

% Write MV to EPKS
write(cont3.bmp241_write, cont3.mvDes(n));
cont3.OPCprev = cont3.OPC(n);

%%          7: Controller 4, [CRP-341 (Cyclone Recycle Pump)]
cont4.MlOpcEnable = false;
    % ^^^ This be made true to enable any controllers configured in
    %      MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont4.crp341_read));
tempreadPv1 = struct2cell(read(cont4.pv1_read));
tempreadPv2 = struct2cell(read(cont4.pv2_read));
tempreadDv = struct2cell(read(cont4.dv_read));
tempreadSp = struct2cell(read(cont4.sp_read));
tempOpcState = struct2cell(read(cont4.opcEnable));
tempOpcVarA = struct2cell(read(cont4.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont4.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont4.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont4.OpcVarD_read));

cont4.crp341(n) = cell2mat(tempreadMvOut(2));
cont4.pv1(n) = cell2mat(tempreadPv1(2));
cont4.pv2(n) = cell2mat(tempreadPv2(2));
cont4.dv(n) = cell2mat(tempreadDv(2));
cont4.sp(n) = cell2mat(tempreadSp(2));
cont4.OPC(n) = cell2mat(tempOpcState(2));
cont4.OpcVarA = cell2mat(tempOpcVarA(2));
cont4.OpcVarB = cell2mat(tempOpcVarB(2));
cont4.OpcVarC = cell2mat(tempOpcVarC(2));
cont4.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont4.OPC(n) == 1 && cont4.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %      *** See Line 133

% ----- CRP-341 Controller Goes Here -----
% #####
% #####
    cont4.K = cont4.OpcVarA;
    cont4.ti = cont4.OpcVarB;

    cont4.err(n) = cont4.sp(n) - cont4.pv1(n);
    tempmvDes = cont4.mvDes(n-1) + cont4.K*((1+(dt/cont4.ti))*...
        *cont4.err(n) - cont4.err(n-1));
    if tempmvDes > 105
        cont4.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont4.mvDes(n) = -5;
    else
        cont4.mvDes(n) = tempmvDes;
    end

% #####
% #####
% -----
elseif (cont4.OPC(n) == 1 && cont4.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont4.opcEnable, 0); % < Disable OPC in EPKS
    cont4.err(n) = 0;

```

```

    cont4.mvDes(n) = cont4.crp341(n);
else
    % \\\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont4.err(n) = 0;
    cont4.mvDes(n) = cont4.crp341(n);
end

% Write MV to EPKS
write(cont4.crp341_write, cont4.mvDes(n));
cont4.OPCprev = cont4.OPC(n);

%%      8: Controller 5, [CUFP-361 (Cyclone Underflow Pump)]
cont5.M10pcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%      MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont5.cufp361_read));
tempreadPv1 = struct2cell(read(cont5.pv1_read));
tempreadPv2 = struct2cell(read(cont5.pv2_read));
tempreadDv = struct2cell(read(cont5.dv_read));
tempreadSp = struct2cell(read(cont5.sp_read));
tempOpcState = struct2cell(read(cont5.opcEnable));
tempOpcVarA = struct2cell(read(cont5.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont5.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont5.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont5.OpcVarD_read));

cont5.cufp361(n) = cell2mat(tempreadMvOut(2));
cont5.pv1(n) = cell2mat(tempreadPv1(2));
cont5.pv2(n) = cell2mat(tempreadPv2(2));
cont5.dv(n) = cell2mat(tempreadDv(2));
cont5.sp(n) = cell2mat(tempreadSp(2));
cont5.OPC(n) = cell2mat(tempOpcState(2));
cont5.OpcVarA = cell2mat(tempOpcVarA(2));
cont5.OpcVarB = cell2mat(tempOpcVarB(2));
cont5.OpcVarC = cell2mat(tempOpcVarC(2));
cont5.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont5.OPC(n) == 1 && cont5.M10pcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %      *** See Line 133

    % ----- CUFP-361 Controller Goes Here -----
    % #####
    % #####
    cont5.K = cont5.OpcVarA;
    cont5.ti = cont5.OpcVarB;

    cont5.err(n) = cont5.sp(n) - cont5.pv1(n);
    tempmvDes = cont5.mvDes(n-1) + cont5.K*((1+(dt/cont5.ti))...
        *cont5.err(n) - cont5.err(n-1));

    if tempmvDes > 105
        cont5.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont5.mvDes(n) = -5;
    else
        cont5.mvDes(n) = tempmvDes;
    end

    % #####
    % #####
    % -----
elseif (cont5.OPC(n) == 1 && cont5.M10pcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont5.opcEnable, 0); % < Disable OPC in EPKS
    cont5.err(n) = 0;
    cont5.mvDes(n) = cont5.cufp361(n);
else
    % \\\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont5.err(n) = 0;
    cont5.mvDes(n) = cont5.cufp361(n);

```

```

end

% Write MV to EPKS
write(cont5.cufp361_write, cont5.mvDes(n));
cont5.OPCprev = cont5.OPC(n);

%%          9: Controller 6, [LUF-421 (Lamella Underflow Pump)]

%%          10: Controller 7, [FDP-521 (Feed Disturbance Pump)]
cont7.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
% MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont7.fdp521_read));
tempreadPv1 = struct2cell(read(cont7.pv1_read));
tempreadPv2 = struct2cell(read(cont7.pv2_read));
tempreadDv = struct2cell(read(cont7.dv_read));
tempreadSp = struct2cell(read(cont7.sp_read));
tempOpcState = struct2cell(read(cont7.opcEnable));
tempOpcVarA = struct2cell(read(cont7.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont7.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont7.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont7.OpcVarD_read));

cont7.fdp521(n) = cell2mat(tempreadMvOut(2));
cont7.pv1(n) = cell2mat(tempreadPv1(2));
cont7.pv2(n) = cell2mat(tempreadPv2(2));
cont7.dv(n) = cell2mat(tempreadDv(2));
cont7.sp(n) = cell2mat(tempreadSp(2));
cont7.OPC(n) = cell2mat(tempOpcState(2));
cont7.OpcVarA = cell2mat(tempOpcVarA(2));
cont7.OpcVarB = cell2mat(tempOpcVarB(2));
cont7.OpcVarC = cell2mat(tempOpcVarC(2));
cont7.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont7.OPC(n) == 1 && cont7.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    % *** See Line 133

% ----- FDP-521 Controller Goes Here -----
% #####
% #####
    cont7.K = cont7.OpcVarA;
    cont7.ti = cont7.OpcVarB;

    cont7.err(n) = cont7.sp(n) - cont7.pv1(n);
    tempmvDes = cont7.mvDes(n-1) + cont7.K*((1+(dt/cont7.ti))...
        *cont7.err(n) - cont7.err(n-1));
    if tempmvDes > 105
        cont7.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont7.mvDes(n) = -5;
    else
        cont7.mvDes(n) = tempmvDes;
    end

% #####
% #####
% -----
elseif (cont7.OPC(n) == 1 && cont7.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont7.opcEnable, 0); % < Disable OPC in EPKS
    cont7.err(n) = 0;
    cont7.mvDes(n) = cont7.fdp521(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont7.err(n) = 0;
    cont7.mvDes(n) = cont7.fdp521(n);
end

```

```

% Write MV to EPKS
write(cont7.fdp521_write, cont7.mvDes(n));
cont7.OPCprev = cont7.OPC(n);

%%          11: Controller 8, [FCV-541 (NLT input)]
cont8.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%   MATLAB, else 0 will be written to OPC server.

%   updating controller Variables
tempreadMvOut = struct2cell(read(cont8.fcv541_read));
tempreadPv1 = struct2cell(read(cont8.pv1_read));
tempreadPv2 = struct2cell(read(cont8.pv2_read));
tempreadDv = struct2cell(read(cont8.dv_read));
tempreadSp = struct2cell(read(cont8.sp_read));
tempOpcState = struct2cell(read(cont8.opcEnable));
tempOpcVarA = struct2cell(read(cont8.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont8.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont8.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont8.OpcVarD_read));

cont8.fcv541(n) = cell2mat(tempreadMvOut(2));
cont8.pv1(n) = cell2mat(tempreadPv1(2));
cont8.pv2(n) = cell2mat(tempreadPv2(2));
cont8.dv(n) = cell2mat(tempreadDv(2));
cont8.sp(n) = cell2mat(tempreadSp(2));
cont8.OPC(n) = cell2mat(tempOpcState(2));
cont8.OpcVarA = cell2mat(tempOpcVarA(2));
cont8.OpcVarB = cell2mat(tempOpcVarB(2));
cont8.OpcVarC = cell2mat(tempOpcVarC(2));
cont8.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont8.OPC(n) == 1 && cont8.MlOpcEnable == 1)
%   ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
%   *** See Line 133

% ----- FCV-541 Controller Goes Here -----
% #####
% #####
cont8.K = cont8.OpcVarA;
cont8.ti = cont8.OpcVarB;

cont8.err(n) = cont8.sp(n) - cont8.pv1(n);
tempmvDes = cont8.mvDes(n-1) + cont8.K*((1+(dt/cont8.ti))*...
    *cont8.err(n) - cont8.err(n-1));
if tempmvDes > 105
    cont8.mvDes(n) = 105;
elseif tempmvDes < -5
    cont8.mvDes(n) = -5;
else
    cont8.mvDes(n) = tempmvDes;
end

% #####
% #####
% -----
elseif (cont8.OPC(n) == 1 && cont8.MlOpcEnable == 0)
%   ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
%   disable OPC in EPKS

write(cont8.opcEnable, 0); % < Disable OPC in EPKS
cont8.err(n) = 0;
cont8.mvDes(n) = cont8.fcv541(n);
else
%   \/\ If OPC disabled in EPKS and ML make error zero and make
%   MATLAB desired MV track EPKS outerloop MV.

cont8.err(n) = 0;
cont8.mvDes(n) = cont8.fcv541(n);
end

% Write MV to EPKS
write(cont8.fcv541_write, cont8.mvDes(n));
cont8.OPCprev = cont8.OPC(n);

%%          12: Controller 9, [NTP-561 (Needle Tank Underflow Pump)]
cont9.MlOpcEnable = false;
%   ^^^ This be made true to enable any controllers configured in

```

```

% MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont9.ntp561_read));
tempreadPv1 = struct2cell(read(cont9.pv1_read));
tempreadPv2 = struct2cell(read(cont9.pv2_read));
tempreadDv = struct2cell(read(cont9.dv_read));
tempreadSp = struct2cell(read(cont9.sp_read));
tempOpcState = struct2cell(read(cont9.opcEnable));
tempOpcVarA = struct2cell(read(cont9.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont9.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont9.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont9.OpcVarD_read));

cont9.ntp561(n) = cell2mat(tempreadMvOut(2));
cont9.pv1(n) = cell2mat(tempreadPv1(2));
cont9.pv2(n) = cell2mat(tempreadPv2(2));
cont9.dv(n) = cell2mat(tempreadDv(2));
cont9.sp(n) = cell2mat(tempreadSp(2));
cont9.OPC(n) = cell2mat(tempOpcState(2));
cont9.OpcVarA = cell2mat(tempOpcVarA(2));
cont9.OpcVarB = cell2mat(tempOpcVarB(2));
cont9.OpcVarC = cell2mat(tempOpcVarC(2));
cont9.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont9.OPC(n) == 1 && cont9.M10pcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    % *** See Line 133

% ----- NTP-561 Controller Goes Here -----
% #####
% #####
    cont9.K = cont9.OpcVarA;
    cont9.ti = cont9.OpcVarB;

    cont9.err(n) = cont9.sp(n) - cont9.pv1(n);
    tempmvDes = cont9.mvDes(n-1) + cont9.K*((1+(dt/cont9.ti))...
        *cont9.err(n) - cont9.err(n-1));

    if tempmvDes > 105
        cont9.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont9.mvDes(n) = -5;
    else
        cont9.mvDes(n) = tempmvDes;
    end

% #####
% #####
% -----
elseif (cont9.OPC(n) == 1 && cont9.M10pcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %     disable OPC in EPKS

    write(cont9.opcEnable, 0); % < Disable OPC in EPKS
    cont9.err(n) = 0;
    cont9.mvDes(n) = cont9.ntp561(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %     MATLAB desired MV track EPKS outerloop MV.

    cont9.err(n) = 0;
    cont9.mvDes(n) = cont9.ntp561(n);
end

% Write MV to EPKS
write(cont9.ntp561_write, cont9.mvDes(n));
cont9.OPCprev = cont9.OPC(n);

%%          13: Controller 10, [FCV-570 (CSTR1 fluid input)]
cont10.M10pcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%     MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont10.fcv570_read));
tempreadPv1 = struct2cell(read(cont10.pv1_read));
tempreadPv2 = struct2cell(read(cont10.pv2_read));
tempreadDv = struct2cell(read(cont10.dv_read));

```

```

tempreadSp = struct2cell(read(cont10.sp_read));
tempOpcState = struct2cell(read(cont10.opcEnable));
tempOpcVarA = struct2cell(read(cont10.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont10.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont10.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont10.OpcVarD_read));

cont10.fcv541(n) = cell2mat(tempreadMvOut(2));
cont10.pv1(n) = cell2mat(tempreadPv1(2));
cont10.pv2(n) = cell2mat(tempreadPv2(2));
cont10.dv(n) = cell2mat(tempreadDv(2));
cont10.sp(n) = cell2mat(tempreadSp(2));
cont10.OPC(n) = cell2mat(tempOpcState(2));
cont10.OpcVarA = cell2mat(tempOpcVarA(2));
cont10.OpcVarB = cell2mat(tempOpcVarB(2));
cont10.OpcVarC = cell2mat(tempOpcVarC(2));
cont10.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont10.OPC(n) == 1 && cont10.M1opcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %     *** See Line 133

% ----- FCV-570 Controller Goes Here -----
% #####
% #####
    cont10.K = cont10.OpcVarA;
    cont10.ti = cont10.OpcVarB;

    cont10.err(n) = cont10.sp(n) - cont10.pv1(n);
    tempmvDes = cont10.mvDes(n-1) + cont10.K*((1+(dt/cont10.ti))...
        *cont10.err(n) - cont10.err(n-1));
    if tempmvDes > 105
        cont10.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont10.mvDes(n) = -5;
    else
        cont10.mvDes(n) = tempmvDes;
    end

% #####
% #####
% -----
elseif (cont10.OPC(n) == 1 && cont10.M1opcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %     disable OPC in EPKS

    write(cont10.opcEnable, 0); % < Disable OPC in EPKS
    cont10.err(n) = 0;
    cont10.mvDes(n) = cont10.fcv541(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %     MATLAB desired MV track EPKS outerloop MV.

    cont10.err(n) = 0;
    cont10.mvDes(n) = cont10.fcv570(n);
end

% Write MV to EPKS
write(cont10.fcv570_write, cont10.mvDes(n));
cont10.OPCprev = cont10.OPC(n);

%%
    14: Controller 11, [FCV-574 (NT to BMT)] *OOC*
    % This asset is currently out of commission, leave empty until
    % asset is operational.
    %
    %
    %
    %
    15: Controller 12, [FCV-662 (CSTR1 Steam In)] *OOC*
    % This asset is currently out of commission, leave empty until
    % asset is operational.
    %
    %
    %
    %
    16: Controller 13, [FCV-642 (CSTR2 Steam In)]

```

```

cont13.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%   MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont13.fcv642_read));
tempreadPv1 = struct2cell(read(cont13.pv1_read));
tempreadPv2 = struct2cell(read(cont13.pv2_read));
tempreadDv = struct2cell(read(cont13.dv_read));
tempreadSp = struct2cell(read(cont13.sp_read));
tempOpcState = struct2cell(read(cont13.opcEnable));
tempOpcVarA = struct2cell(read(cont13.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont13.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont13.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont13.OpcVarD_read));

cont13.fcv642(n) = cell2mat(tempreadMvOut(2));
cont13.pv1(n) = cell2mat(tempreadPv1(2));
cont13.pv2(n) = cell2mat(tempreadPv2(2));
cont13.dv(n) = cell2mat(tempreadDv(2));
cont13.sp(n) = cell2mat(tempreadSp(2));
cont13.OPC(n) = cell2mat(tempOpcState(2));
cont13.OpcVarA = cell2mat(tempOpcVarA(2));
cont13.OpcVarB = cell2mat(tempOpcVarB(2));
cont13.OpcVarC = cell2mat(tempOpcVarC(2));
cont13.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont13.OPC(n) == 1 && cont13.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %   *** See Line 133

    % ----- FCV-642 Controller Goes Here -----
    % #####
    % #####
    cont13.K = cont13.OpcVarA;
    cont13.ti = cont13.OpcVarB;

    cont13.err(n) = cont13.sp(n) - cont13.pv1(n);
    tempmvDes = cont13.mvDes(n-1) + cont13.K*((1+(dt/cont13.ti))...
        *cont13.err(n) - cont13.err(n-1));
    if tempmvDes > 105
        cont13.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont13.mvDes(n) = -5;
    else
        cont13.mvDes(n) = tempmvDes;
    end

    % #####
    % #####
    % -----
elseif (cont13.OPC(n) == 1 && cont13.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %   disable OPC in EPKS

    write(cont13.opcEnable, 0); % < Disable OPC in EPKS
    cont13.err(n) = 0;
    cont13.mvDes(n) = cont13.fcv642(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %   MATLAB desired MV track EPKS outerloop MV.

    cont13.err(n) = 0;
    cont13.mvDes(n) = cont13.fcv642(n);
end

% Write MV to EPKS
write(cont13.fcv642_write, cont13.mvDes(n));
cont13.OPCprev = cont13.OPC(n);

%%          17: Controller 14, [FCV-662 (CSTR3 Steam In)]
cont14.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%   MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont14.fcv662_read));
tempreadPv1 = struct2cell(read(cont14.pv1_read));

```

```

tempreadPv2 = struct2cell(read(cont14.pv2_read));
tempreadDv = struct2cell(read(cont14.dv_read));
tempreadSp = struct2cell(read(cont14.sp_read));
tempOpcState = struct2cell(read(cont14.opcEnable));
tempOpcVarA = struct2cell(read(cont14.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont14.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont14.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont14.OpcVarD_read));

cont14.fcv662(n) = cell2mat(tempreadMvOut(2));
cont14.pv1(n) = cell2mat(tempreadPv1(2));
cont14.pv2(n) = cell2mat(tempreadPv2(2));
cont14.dv(n) = cell2mat(tempreadDv(2));
cont14.sp(n) = cell2mat(tempreadSp(2));
cont14.OPC(n) = cell2mat(tempOpcState(2));
cont14.OpcVarA = cell2mat(tempOpcVarA(2));
cont14.OpcVarB = cell2mat(tempOpcVarB(2));
cont14.OpcVarC = cell2mat(tempOpcVarC(2));
cont14.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont14.OPC(n) == 1 && cont14.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    % *** See Line 133

% ----- FCV-662 Controller Goes Here -----
% #####
% #####
    cont14.K = cont14.OpcVarA;
    cont14.ti = cont14.OpcVarB;

    cont14.err(n) = cont14.sp(n) - cont14.pv1(n);
    tempmvDes = cont14.mvDes(n-1) + cont14.K*((1+(dt/cont14.ti))...
        *cont14.err(n) - cont14.err(n-1));

    if tempmvDes > 105
        cont14.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont14.mvDes(n) = -5;
    else
        cont14.mvDes(n) = tempmvDes;
    end

% #####
% #####
% -----
elseif (cont14.OPC(n) == 1 && cont14.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %      disable OPC in EPKS

    write(cont14.opcEnable, 0); % < Disable OPC in EPKS
    cont14.err(n) = 0;
    cont14.mvDes(n) = cont14.fcv662(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outerloop MV.

    cont14.err(n) = 0;
    cont14.mvDes(n) = cont14.fcv662(n);
end

% Write MV to EPKS
write(cont14.fcv662_write, cont14.mvDes(n));
cont14.OPCprev = cont14.OPC(n);

%%
    18: Controller 15, [FCV-688 (System Output)]
cont15.MlOpcEnable = true;
    % ^^^ This be made true to enable any controllers configured in
    %      MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont15.fcv688_read));
tempreadPv1 = struct2cell(read(cont15.pv1_read));
tempreadPv2 = struct2cell(read(cont15.pv2_read));
tempreadDv = struct2cell(read(cont15.dv_read));
tempreadSp = struct2cell(read(cont15.sp_read));
tempOpcState = struct2cell(read(cont15.opcEnable));
tempOpcVarA = struct2cell(read(cont15.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont15.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont15.OpcVarC_read));

```



```

tempOpcVarD = struct2cell(read(cont15.OpcVarD_read));

cont15.fcv688(n) = cell2mat(tempreadMvOut(2));
cont15.pv1(n) = cell2mat(tempreadPv1(2));
cont15.pv2(n) = cell2mat(tempreadPv2(2));
cont15.dv(n) = cell2mat(tempreadDv(2));
cont15.sp(n) = cell2mat(tempreadSp(2));
cont15.OPC(n) = cell2mat(tempOpcState(2));
cont15.OpcVarA = cell2mat(tempOpcVarA(2));
cont15.OpcVarB = cell2mat(tempOpcVarB(2));
cont15.OpcVarC = cell2mat(tempOpcVarC(2));
cont15.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont15.OPC(n) == 1 && cont15.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    %     *** See Line 133

    % ----- FCV-688 Controller Goes Here -----
    % #####
    % #####
    cont15.K = cont15.OpcVarA;
    cont15.ti = cont15.OpcVarB;

    cont15.err(n) = cont15.sp(n) - cont15.pv1(n);
    tempmvDes = cont15.mvDes(n-1) + cont15.K*((1+(dt/cont15.ti))*...
        *cont15.err(n) - cont15.err(n-1));
    if tempmvDes > 105
        cont15.mvDes(n) = 105;
    elseif tempmvDes < -5
        cont15.mvDes(n) = -5;
    else
        cont15.mvDes(n) = tempmvDes;
    end

    % #####
    % #####
    % -----
elseif (cont15.OPC(n) == 1 && cont15.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %     disable OPC in EPKS

    write(cont15.opcEnable, 0); % < Disable OPC in EPKS
    cont15.err(n) = 0;
    cont15.mvDes(n) = cont15.fcv688(n);
else
    % \\\ If OPC disabled in EPKS and ML make error zero and make
    %     MATLAB desired MV track EPKS outerloop MV.

    cont15.err(n) = 0;
    cont15.mvDes(n) = cont15.fcv688(n);
end

% Write MV to EPKS
write(cont15.fcv688_write, cont15.mvDes(n));
cont15.OPCprev = cont15.OPC(n);

%%          19: Controller 16, [FCV-690 (CSTR3 Reflux)]
cont16.MlOpcEnable = false;
    % ^^^ This be made true to enable any controllers configured in
    %     MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont16.fcv690_read));
tempreadPv1 = struct2cell(read(cont16.pv1_read));
tempreadPv2 = struct2cell(read(cont16.pv2_read));
tempreadDv = struct2cell(read(cont16.dv_read));
tempreadSp = struct2cell(read(cont16.sp_read));
tempOpcState = struct2cell(read(cont16.opcEnable));
tempOpcVarA = struct2cell(read(cont16.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont16.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont16.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont16.OpcVarD_read));

cont16.fcv690(n) = cell2mat(tempreadMvOut(2));
cont16.pv1(n) = cell2mat(tempreadPv1(2));
cont16.pv2(n) = cell2mat(tempreadPv2(2));
cont16.dv(n) = cell2mat(tempreadDv(2));
cont16.sp(n) = cell2mat(tempreadSp(2));

```

```

cont16.OPC(n) = cell2mat(tempOpcState(2));
cont16.OpcVarA = cell2mat(tempOpcVarA(2));
cont16.OpcVarB = cell2mat(tempOpcVarB(2));
cont16.OpcVarC = cell2mat(tempOpcVarC(2));
cont16.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont16.OPC(n) == 1 && cont16.MlOpcEnable == 1)
    % ^^^ If OPC is enabled in EPKS && OPC is also enabled in MATLAB
    % *** See Line 133

% ----- FCV-690 Controller Goes Here -----
% #####
% #####
cont16.K = cont16.OpcVarA;
cont16.ti = cont16.OpcVarB;

cont16.err(n) = cont16.sp(n) - cont16.pv1(n);
tempmvDes = cont16.mvDes(n-1) + cont16.K*((1+(dt/cont16.ti))...
    *cont16.err(n) - cont16.err(n-1));

if tempmvDes > 105
    cont16.mvDes(n) = 105;
elseif tempmvDes < -5
    cont16.mvDes(n) = -5;
else
    cont16.mvDes(n) = tempmvDes;
end

% #####
% #####
% -----
elseif (cont16.OPC(n) == 1 && cont16.MlOpcEnable == 0)
    % ^^^ if OPC is enabled in EPKS BUT is not enabled in MATLAB,
    %     disable OPC in EPKS

    write(cont16.opcEnable, 0); % < Disable OPC in EPKS
    cont16.err(n) = 0;
    cont16.mvDes(n) = cont16.fcv690(n);
else
    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %     MATLAB desired MV track EPKS outerloop MV.

    cont16.err(n) = 0;
    cont16.mvDes(n) = cont16.fcv690(n);
end

% Write MV to EPKS
write(cont16.fcv690_write, cont16.mvDes(n));
cont16.OPCprev = cont16.OPC(n);

%%          20: Controller 17, [PP-681 (Product Pump)]
cont17.MlOpcEnable = false;
% ^^^ This be made true to enable any controllers configured in
%     MATLAB, else 0 will be written to OPC server.

% updating controller Variables
tempreadMvOut = struct2cell(read(cont17.pp681_read));
tempreadPv1 = struct2cell(read(cont17.pv1_read));
tempreadPv2 = struct2cell(read(cont17.pv2_read));
tempreadDv = struct2cell(read(cont17.dv_read));
tempreadSp = struct2cell(read(cont17.sp_read));
tempOpcState = struct2cell(read(cont17.opcEnable));
tempOpcVarA = struct2cell(read(cont17.OpcVarA_read));
tempOpcVarB = struct2cell(read(cont17.OpcVarB_read));
tempOpcVarC = struct2cell(read(cont17.OpcVarC_read));
tempOpcVarD = struct2cell(read(cont17.OpcVarD_read));

cont17.pp681(n) = cell2mat(tempreadMvOut(2));
cont17.pv1(n) = cell2mat(tempreadPv1(2));
cont17.pv2(n) = cell2mat(tempreadPv2(2));
cont17.dv(n) = cell2mat(tempreadDv(2));
cont17.sp(n) = cell2mat(tempreadSp(2));
cont17.OPC(n) = cell2mat(tempOpcState(2));
cont17.OpcVarA = cell2mat(tempOpcVarA(2));
cont17.OpcVarB = cell2mat(tempOpcVarB(2));
cont17.OpcVarC = cell2mat(tempOpcVarC(2));
cont17.OpcVarD = cell2mat(tempOpcVarD(2));

if (cont17.OPC(n) == 1 && cont17.MlOpcEnable == 1)

```

